MICRORAMA: Microsecond Oblivious Access to Gigabytes of Data

Anonymous Submission

***Abstract*—Trusted execution environments (TEEs) protect the contents of memory, but secret-dependent memory accesses can still leak sensitive information through side channels. Oblivious RAM (ORAM) complements a TEE by hiding which memory locations a program accesses. While recent systems such as H2O2RAM (USENIX'25) and SONIC (USENIX'26) have substantially improved ORAM throughput inside TEEs, they remain one to two orders of magnitude slower than non-oblivious implementations.**

We present MICRORAMA, a highly efficient ORAM and oblivious map (OMAP) for commodity x86 TEEs with AVX-512. On a 16-core Intel TDX virtual machine and with 2^{27} 8-byte words, our deamortized hierarchical ORAM serves up to 1.04 M requests per second one at a time, $12.8\times$ the throughput of H2O2RAM and 70% of the throughput of a non-oblivious `std::unordered_map`. When requests arrive at half of this rate, we achieve an average response time of 0.93 μs and a 99th-percentile response time of 1.4 μs. In comparison, H2O2RAM's average and 99th-percentile response times at half of its own throughput are 4.0 and 90 s, respectively, because its rebuilds block the requests that arrive in the meantime. When requests arrive in batches, we serve 3.36 M requests per second on the same machine, $70\times$ the throughput of SONIC. We use 10.0 GiB of memory, against 22.1 GiB for H2O2RAM and 27.7 GiB for SONIC.

For values of 128 bytes or more, we switch to a hybrid that stores the values in a tree-based ORAM and their positions in the hierarchical ORAM. With 1 KiB values, the hybrid serves $5.4\times$ as many requests as the hierarchical ORAM alone and reduces memory by 44%.

We target a failure probability below 2^{-64} over 2^{32} accesses, and we prove in Lean that a model of our hierarchical ORAM meets this target, that every lookup returns the newest value, and that its hash-table accesses hide the requested keys. Unlike H2O2RAM, we also hide repeated lookups of an absent key.

1. Introduction

Trusted execution environments (TEEs) allow a cloud service to process sensitive data while protecting those data from the host operating system. Memory encryption alone, however, does not hide which locations an enclave accesses. Page faults, processor caches, and other side channels can expose secret-dependent memory-access patterns even when the values stored at those locations remain encrypted [1], [2], [3]. Oblivious RAM (ORAM) [4], [5] addresses this leakage by making the observable memory trace independent of the logical addresses requested. We assume that the adversary observes the address of every memory access inside the enclave, including accesses served by the processor caches, and the running time of every instruction. The adversary cannot observe the contents of registers. This model is known as *double obliviousness* in prior work [6], [7], [8].

While a standard ORAM only serves reads and writes to the dense addresses $0, \ldots, N-1$, we also support sparse keys drawn from a much larger domain. Prior work calls this abstraction an oblivious map (OMAP) [8]: it supports key–value lookups and updates while hiding access patterns. Goldreich and Ostrovsky's hierarchical ORAM [5], [9] provides a direct way to support these operations: it stores records in *levels*, hash tables of increasing size that index application keys. An access probes the tables from smallest to largest and writes the requested record back to the smallest one. The smaller tables therefore fill and must periodically be rebuilt into larger ones. H2O2RAM [10], a recent practical instantiation of hierarchical ORAM, demonstrates that this approach can achieve low *amortized* access cost inside a TEE.

Low amortized cost, however, does not ensure low latency for each request. H2O2RAM stops serving requests while it rebuilds its hash tables, and its largest table can hold every record in the ORAM. A rebuild of that table delays every request that arrives in the meantime, even when the average cost per access is small. To avoid these pauses, Ostrovsky and Shoup [11] proposed a deamortization scheme that rebuilds the tables gradually while requests continue. The old tables must then remain queryable until their replacements are built, so both occupy memory at once. This creates a practical tradeoff between pauses and memory in prior deamortization schedules [11], [12], [13].

Tree-based ORAM [14], [15], [16] offers an alternative. [Tianyao: TODO: Establish the memory advantage over a specified hierarchical baseline for the relevant large-entry configurations, counting tree capacity, position maps, stashes, and rebuild storage consistently. Evidence so far (our own constructions, library allocation over logical data, 2^{20} records): the hierarchical ORAM needs about $4.9\times$ at every value size, the tree $3.4\times$ at 128 bytes and $2.7\times$ at 4 KiB, and H2O2RAM's process 14 to $17\times$ up to 1 KiB (Figure 8).] Schemes such as Path ORAM [14] avoid periodic hash-table rebuilds. Each access, however, must first obliviously look up the record's tree position in a position map, whose entries are small [14]. This lookup has a cost largely independent of the data-entry size, so data entries must be sufficiently large to amortize it [14], [15].

We present MICRORAMA, which combines the two approaches on x86 TEEs with AVX-512. For values smaller than 128 bytes, MICRORAMA uses a hierarchical ORAM that rebuilds its hash tables in the background without pausing requests, yet needs less memory than H2O2RAM. For larger values, MICRORAMA stores the values in Path ORAM and keeps their tree positions in this hierarchical ORAM.

1.1. Our Contributions

A fast ORAM and OMAP library for TEEs. We implement MicroRAMa as a C++ library for commodity x86 processors with AVX-512. It provides ORAM and OMAP interfaces, the latter with lookup, insertion, update, and erasure by arbitrary keys. For 8-byte values on Intel TDX, MicroRAMa achieves 14.2 to 15.4× the throughput of H2O2RAM with up to 2^{21} records. With 2^{27} records (1.5 GiB), it serves up to 1.04 M requests per second, 12.8× the throughput of H2O2RAM and 70% of the throughput of a non-oblivious `std::unordered_map`. When requests arrive at half of this rate, MicroRAMa achieves an average response time of 0.93 μs and a P99 response time of 1.4 μs. In comparison, H2O2RAM's average and P99 response times at half of its own throughput are 4.0 and 90 s, respectively, because its rebuilds block the requests that arrive in the meantime. MicroRAMa also uses less memory than H2O2RAM, 10.0 against 22.1 GiB. For 1 KiB values, MicroRAMa achieves 5.0× the throughput of SONIC without a position map, as in SONIC's own measurements, and 8.8× once SONIC's position map is made oblivious. Our measurements include the oblivious position-map lookup of every access (Section 5). [Tianyao: The memory comparison is whole-process peak resident memory, excluding H2O2RAM's 115 GiB preparation; use matched allocations.]

Vectorized hierarchical rebuilding. We accelerate hierarchical rebuilding by routing records through a *butterfly network* with AVX-512. For n records, this replaces H2O2RAM's $\Theta(\log^2 n)$ bitonic sorting layers [17] with $\Theta(\log n)$ routing stages. Each stage consists of *merge-splits*, which split the records of two buckets by a destination bit. Prior work [18], [19], [20] implements these with expensive oblivious sorting or compaction over large buckets. We instead store 64-slot buckets byte-sliced, so one AVX-512 register holds the same byte of every slot. A register permutation routes that byte for all 64 slots without revealing their destinations, because it accesses no memory. Small buckets overflow often, so each stage has a fixed-capacity stash adapted from NanoGRAM's shared overflow pile [21]. A final sweep places these records in their destination buckets when space remains (Section 2).

On our Intel evaluation machine, this network builds a hash table 56 to 258× faster than H2O2RAM's bitonic sort (Section 5.4). [Tianyao: The measured network covers every routing stage, permutation planning, and the residual stashes, but not duplicate removal or the final sweep that recovers residuals. Time a complete merge before calling this a rebuild speedup.]

Vectorized rebuilds are cheap relative to lookups, which still make dependent random accesses to several tables. Following MegaBlocks [22], we trade more rebuild work for fewer lookup probes by increasing the branching factor, the capacity ratio between successive levels.

Deamortized rebuilding with lower space overhead. Running rebuilds on background workers avoids request pauses. With a large branching factor and optimizations to the largest table, MicroRAMa allocates 6.5× the space of the raw data with 2^{27} records (Section 5.2). [Tianyao: TODO: Check the allocation accounting for the claim that region reuse provides a smaller additional saving, especially at large branching factors; do not report an incremental percentage without evidence.] [Tianyao: Measured at 2^{27} records with 8-byte values, MicroRAMa allocates 6.5× the logical data, 99% of it in main tables, matching the library's model to 0.1% (Section 5.2). The matched H2O2RAM allocation is still missing; this ratio is not total memory overhead.] Moving rebuilds off the request thread also raises throughput: with three background workers, MicroRAMa serves 1.98× (Intel) and 1.24× (AMD) as many requests as when the request thread runs every merge itself (Section 5.4). [Tianyao: With the same single core, running the merges in pieces on the request thread is 16% slower than running them whole on Intel (422 vs. 504 K requests/s at 2^{27}; 17% on AMD), so the gain needs the extra cores. Decide whether to say so here.]

Hiding repeated misses. Following earlier theoretical work [23], [24], MicroRAMa appends a tombstone, a record marking the key absent, after each unsuccessful lookup. This hides repeated lookups of absent keys, which H2O2RAM exposes by probing the same buckets again [25] (Section 2.3).

A complementary tree ORAM for large entries. For large entries, MicroRAMa stores application payloads in a tree adapted from Path ORAM [14] and uses our small-entry hierarchical OMAP as its *position map*, which records each block's current tree position. This hybrid keeps large payloads out of the hash tables and their rebuilds, which further reduces storage overhead.

We also vectorize the tree's *path eviction*, which rearranges the blocks along each accessed path [14]. The original Path ORAM paper assumes a client-server model, in which the client performs this step in a small trusted memory. In a TEE, the eviction must itself be oblivious. Prior work implements it with either naive linear scans [7] or bitonic sorting [8]. We instead rearrange the blocks on the path with AVX-512 permutation instructions, using a byte-slicing technique similar to that of our merge-split. For large entries, we further parallelize the eviction by assigning each worker a disjoint range of payload bytes.

On our evaluation machines, the tree construction becomes faster than the hierarchical ORAM between 64- and 128-byte values and also uses less memory, so MicroRAMa selects it from 128-byte values on (Section 5.3). [Tianyao: On AMD the crossover is 67–73 B, so 96-byte values run on the slower construction; decide the threshold, and recheck with batches.]

1.2. Related Work

Hierarchical ORAM. Ostrovsky [5], [9] proposed hierarchical ORAM, the first ORAM with polylogarithmic overhead. Later work reduced its amortized overhead. Kushilevitz, Lu, and Ostrovsky [23] reach $O(\log^2 N/\log\log N)$, PanORAMa [26] reaches $O(\log N \log\log N)$, and OptORAMa [27] reaches $O(\log N)$, which matches the lower bound [28], [29]. OptORAMa's construction, however, relies on sophisticated oblivious hashing, compaction, and routing primitives, so the asymptotic bound does not directly translate into a low-cost implementation.

FutORAMa [30] takes a more concrete approach and shows that a hierarchical construction can outperform optimized Path ORAM [14] in the traditional client–server setting. Its performance relies on a sublinear amount of client memory whose access pattern is assumed to remain private. This

assumption is natural when the client is physically separate from the server, but it does not hold in a doubly-oblivious TEE implementation, where the client memory resides in the enclave and accesses to it must themselves be oblivious.

H2O2RAM. H2O2RAM [10] shows that hierarchical ORAM can nevertheless be practical inside a TEE. Instead of directly implementing the theoretical primitives, H2O2RAM designs several concrete oblivious hash tables and selects among them according to the table size and expected number of accesses. It reports up to $10^3\times$ faster map operations than the tree-based EnigMap [8].

Deamortized hierarchical ORAM. Deamortization of hierarchical ORAM predates modern practical constructions. Table 1 compares the storage, rebuild speed, and lookup cost of Ostrovsky and Shoup's schedule [11] and later schemes [12], [13] with ours.

MegaBlocks. MegaBlocks [22] uses a large branching factor to achieve sublogarithmic I/O overhead when external-memory transfers contain many logical words. It trades additional bulk rebuilding for fewer random lookup transfers. We use the same tradeoff in memory, where vectorized rebuilds are cheap (Section 2.4).

Tree-based ORAM and OMAP. Path ORAM [14] and Ring ORAM [16] are practical tree-based designs. Enclave systems such as Oblix [7], GraphOS [31], and EnigMap [8] provide doubly oblivious access on tree-based constructions. SONIC [32] builds on Ring ORAM and improves throughput through concurrent accesses and parallel eviction.

While tree-based ORAM directly supports dense integer addresses, handling sparse keys efficiently requires an additional mapping. Oblix and EnigMap support sparse keys through self-balancing search trees over ORAM [7], [8], at the cost of key traversal and balance maintenance. [Tianyao: TODO: Verify the asymptotic comparison with ORAM under matched block-size, client-memory, and cost metrics; quantify traversal and balancing costs before claiming a substantial practical penalty.]

2. Accelerating Hierarchical ORAM with Vector Instructions

2.1. Background

Hierarchical ORAM. Our construction follows Goldreich and Ostrovsky's hierarchical ORAM [5], [9]. Its smallest level is a dense array, the *online stash*, which every access scans in full. Larger levels have exponentially increasing capacities and store records in bucket hash tables. A secret hash function assigns each key to one bucket of a table.

An access searches the levels from smallest to largest, scanning the requested key's bucket in each larger level. Once a level returns the record, every later level receives a dummy probe of a random bucket, and the access writes the possibly updated record into the online stash. For a key found in a table, later accesses find this newer copy first, so that table receives no further real lookup for the key between rebuilds, hiding whether the key is accessed again [5], [10].

Records written to the online stash eventually fill it, so scheduled rebuilds merge records into larger levels. The schedule depends only on the number of accesses, and each level is rebuilt with a frequency inversely proportional to its capacity. A rebuild must place each record in the bucket selected by the destination table's hash function without revealing that bucket. Goldreich and Ostrovsky's construction and H2O2RAM [10] perform this placement with oblivious sorting. Sorting by bucket alone does not give every bucket the same size, so the rebuild also pads each bucket with dummy records before the sort and removes the surplus dummies afterward. H2O2RAM's implementation performs both the sort and the removal with bitonic sorts [17], [25].

Butterfly networks and merge-split. We instead rebuild the table by routing each record to its hash bucket through a butterfly network. Because a lookup scans only that bucket, the network's output is the table itself, and no sort needs to follow it.

A butterfly network routes records into $m = 2^h$ buckets that each hold up to Z records.[1] Each record carries a secret *routing tag*, and the low h bits of the tag name its destination bucket. To build a hash table, the tag is the new table's secret hash of the record's key. Each bucket starts about half full, and dummies fill its remaining slots. The network's basic operation, *merge-split*, reads two buckets and writes the records whose tag bit for the current stage is zero to the first output bucket and the others to the second. A merge-split overflows if more than Z records go to one output.

The network visits the same bucket pairs regardless of the tags. Stage b pairs bucket i with bucket $i + 2^b$ whenever bit b of i is zero, and splits the pair by tag bit b. After stage b, the low $b + 1$ bits of each record's bucket index therefore match its tag. After h stages, every record is in its destination bucket. The sequence of bucket pairs depends only on m, so the memory trace reveals nothing as long as each merge-split is itself oblivious.

Butterfly networks underlie several oblivious sorting algorithms [18], [19], [33], but their concrete speedup over bitonic sort remains limited, especially for small entries. One reason is the bucket size. The probability that a bucket overflows decreases exponentially in Z, so at a fixed load, a negligible failure probability requires large buckets. Prior butterfly-based sorts use buckets of at least $Z = 512$ records [18], [33]. A merge-split cannot write a record directly to its output slot, because the location of the write would reveal the record's tag. It instead passes the records of its input buckets through a compaction or sorting network of depth at least $\log Z$. These merge-splits therefore make many passes over the data to resolve each tag bit. A hash-table rebuild that used them would inherit the same cost.

AVX-512 instructions. AVX-512 is an extension of the x86 instruction set, supported by current Intel and AMD server processors, whose instructions operate on 512-bit vector registers. One instruction applies the same operation to every part of a register, for example adding eight pairs of 64-bit integers at once. Prior ORAMs for TEEs have used AVX-512 to speed up the conditional copies and swaps of records [8], [25], [32]: one instruction moves up to 64 bytes of a record, against 8 bytes for a scalar instruction.

1. In general, the network's buckets need not be the hash table's buckets. A rebuild could, for example, route records into larger buckets and then divide each of them into several smaller hash-table buckets. In this work, however, we simplify the scheme by setting both kinds of buckets to the same size.

2.2. Byte-sliced Merge-split

Routing inside registers. Our main observation is that AVX-512 can do more than move records as a whole. If we split each record into bytes, one 512-bit register can hold the same byte of 64 records. One permutation instruction, given an index vector, then rearranges this byte of all the records at once. Applying the same indices to each byte position rearranges the whole records. We therefore use buckets of 64 slots for our butterfly network. A merge-split then loads its two input buckets from fixed locations, rearranges their records in registers, and stores the two outputs at fixed locations. The secret routing affects only the permutation indices, which stay in registers, so the merge-split accesses the same memory locations regardless of where its records go.

Byte-sliced layout. If each record were stored contiguously, a merge-split would have to collect the same byte from 64 records, one load per record, before each permutation. We instead store the buckets themselves *byte-sliced* (Figure 1): the same byte position of all 64 slots is contiguous, so one vector load fetches that byte for every slot. The hash tables stay in this layout, so a merge-split loads its registers and stores its outputs without converting any record. With w bytes per slot, a bucket has w *byte lines*, each one 512-bit vector, and a separate 64-bit *occupancy mask*. The mask identifies real records, and the remaining slots are dummies. Each slot contains a key, value, metadata that can mark the key absent, and routing tag. A whole record occupies one column across the byte lines. A lookup checks all 64 slots of a bucket at once by comparing each key byte line with a register filled with the corresponding byte of the requested key. It then extracts the value with one byte permutation per value line, indexed by the matching slot.

For a merge-split of buckets X and Y, we compute two index vectors, π_0 and π_1, from the current routing-tag bit and the occupancy masks. For each byte position, we load one byte line from each input bucket. The two-source byte permutation `VPERMT2B` selects each of its 64 output bytes from the 128 bytes in these two input registers [34]. Applying π_0 produces one output bucket's byte line, and applying π_1 produces the other's. The same two index vectors serve all w byte lines, so each tag moves with its record. These permutations replace the compaction or sorting network of depth at least $\log Z$ within a conventional merge-split.

We size the tables for 32 records per bucket in expectation, retaining the nominal half load used in prior butterfly-network constructions [18]. Small buckets overflow frequently, so we give each stage a fixed-capacity *residual stash* for the records that do not fit, adapting NanoGRAM's overflow collection [21]. Section 2.4 describes these stashes and their overflow bounds.

Planning the permutations. Output $d \in \{0, 1\}$ keeps the records of X whose tag bit is d in their current slots and fills its free slots, in order, with the corresponding records of Y. Records of Y that do not fit go to the stage's residual stash (Section 2.4). A direct implementation would compress the selected bytes of Y and expand them into the free slots of each output, which takes four compress/expand instructions per byte position across the two outputs. We instead apply one compress and one expand to slot indices, once for each index vector (Figure 2). For each output, *compress* packs the selected slot indices of Y, and *expand* places them in the free output positions, which gives π_d. Apart from computing the masks and indices and handling residuals, a merge-split then needs only two `VPERMT2B` instructions per byte position, one for each output.

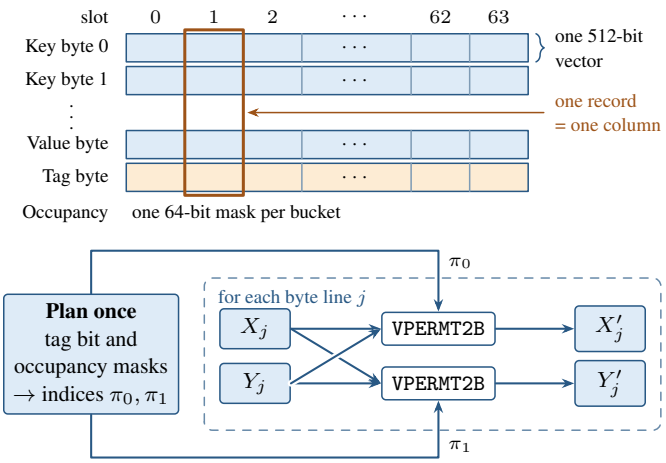


Figure 1: Byte-sliced layout and merge-split. A bucket stores each byte position of its 64 slots in one 512-bit vector, so each row is one byte line and each record is one column. A merge-split of buckets X and Y plans its slot selection once, as the index vectors π_0 and π_1, and applies it to all w byte lines.

Input: buckets X and Y, each with byte lines X_j, Y_j and a 64-bit occupancy mask; tag bit b; stage-b residual stash S_b
Output: X and Y rewritten in place as outputs 0 and 1; overflow appended to S_b

$I \leftarrow (0, \ldots, 63)$; $J \leftarrow (64, \ldots, 127)$ ▷ slots of X, Y
for $d \in \{0, 1\}$ **do** ▷ plan in registers
  $x_d, y_d \leftarrow$ occupied slots of X, Y whose tag bit b is d
  $(Q_d, F_d) \leftarrow \textsc{MatchSlots}(y_d, \neg x_d)$
  $\pi_d \leftarrow \textsc{Expand}(I, F_d, \textsc{Compress}(J, Q_d))$
  $O_d \leftarrow x_d \vee F_d$ ▷ output occupancy
end for
$E \leftarrow (y_0 \wedge \neg Q_0) \vee (y_1 \wedge \neg Q_1)$ ▷ records that do not fit
$\textsc{StashAppend}(S_b, Y, E)$
for each byte line j, in public order **do**
  $r \leftarrow \textsc{Load}(X_j)$; $s \leftarrow \textsc{Load}(Y_j)$
  $\textsc{Store}(X_j, \textsc{Permute2}(r, s, \pi_0, O_0))$
  $\textsc{Store}(Y_j, \textsc{Permute2}(r, s, \pi_1, O_1))$
end for
set the occupancy masks of X and Y to O_0 and O_1

$\textsc{MatchSlots}(q, f)$ pairs the set bits of q with those of f in increasing order, as many as fit, and returns the paired subsets of q and f. $\textsc{Compress}(v, q)$ packs lanes q of v into consecutive low lanes. $\textsc{Expand}(v, f, p)$ writes the packed lanes p into lanes f of v. $\textsc{Permute2}(r, s, \pi, O)$ sets each output lane $i \in O$ to lane $\pi[i]$ of $r \| s$ and zeros the other lanes. $\textsc{StashAppend}(S, Y, E)$ appends slots E of Y to the fixed-capacity stash S and fails if they do not fit. Its memory trace depends only on public capacities: it visits its prescribed storage in S even when E is empty.

Figure 2: Byte-sliced merge-split at stage b.

Constant-time register permutations. Our merge-split's memory accesses are deterministic: it loads and stores the byte lines of its two buckets in a fixed order and never branches on secret data. The secret routing tags reach only

permutation indices and register masks, so they stay hidden as long as the instructions that use them are *constant-time*, that is, take time independent of their operands [35]. Intel guarantees this for the loads, stores, byte comparisons, and mask-register operations that our kernels use, on Ice Lake and later processors only when the data operand independent timing mode is enabled [36], [37]. Its list omits several instructions that our kernels apply to secret data, including the byte permutation, compress, and expand instructions that route records. We assume that these instructions are constant-time, and we test the assumption on both of our evaluation machines (Section 5.1). For every instruction that our kernels apply to secret data, the test times blocks of executions under several classes of secret operands in random order and compares each class with a random reference class. Every instruction passes on the Intel machine, both inside and outside its TDX trust domain: no class differs from the reference by more than 0.0033 cycles per instruction, whereas a one-cycle difference in a control instruction is detected. The test also passes on the AMD machine inside its SEV-SNP guest. [Tianyao: Still open: state the number of variants tested on Zen 5 and its bound, and confirm that the VP2INTERSECT variants are among them; state whether DOITM was enabled on C4 (TDX hides the microcode version). PDEP is microcoded on Zen+ and Zen 2 [38], so state that those processors are out of scope or avoid PDEP there.]

Comparison with prior work. Let n be the number of records, w the bytes per record, and m the number of 64-slot buckets, with $n = \Theta(m)$ at fixed nominal load. Counting logical byte-processing work, the main butterfly network performs $\Theta(mw)$ work per routing stage and has $\log_2 m$ stages, for $\Theta(nw \log n)$ work. A bitonic network with full-record compare-exchanges processes $\Theta(nw \log^2 n)$ bytes across its compare-exchange layers [17]. These counts cover the main networks, not total rebuild work or transfers from main memory. Within them, the logarithmic-factor difference comes from routing records to hash buckets, and fixed-width AVX-512 byte slicing further reduces the cost within each stage. [Tianyao: TODO: Bound index planning, duplicate handling, and residual collection/recovery under the final parameters before extending this comparison to complete rebuilds. State the machine model and failure assumptions explicitly.] In our evaluation, the network builds a hash table of 2^{20} records on one core in 10.6 ms with 8-byte values, whereas H2O2RAM's bitonic sort takes 2.5 s. The build measurements appear in the online full version. [Tianyao: The measured network excludes duplicate removal and the sweep that recovers residuals; time a complete merge before calling this an end-to-end rebuild speedup.]

2.3. Deamortization-friendly Hierarchical ORAM

We now build a hierarchical ORAM whose rebuilds route records with byte-sliced merge-splits. Moreover, we design it to make deamortization easier, namely, to let rebuilds run in the background while requests continue (Section 3).

Read-only hash tables. An access that finds a key in a table writes the possibly updated record to the online stash, so the table still holds an older copy of the key. Some schemes mark this older copy as erased, so that the next rebuild discards it [10], [23], [26], [27], [30], [39]. Others leave the copy in place [5], [11], [12], [24], where the newer copy shadows it: later accesses find the newer copy first and send only dummy probes to the older copy's table.

Marking complicates background rebuilding. A rebuild reads its input tables while accesses continue, so every mark made during the rebuild would have to be logged and replayed on the rebuilt table. We therefore leave older copies in place, so accesses write only to the online stash, and a hash table never changes after it is built. A rebuild then reads a fixed snapshot of its inputs, and accesses and rebuilds never write to the same memory.

Leaving older copies in place also makes writes cheap. Because the newer copy shadows the older one, a write or an erasure need not locate the older copy, so MICRORAMA appends the new record to the online stash without probing the hash tables, which makes these requests faster than lookups. For an erasure, the appended record is a *tombstone*, whose metadata marks the key absent. Skipping the probes, however, reveals that a request is a write. An application that must hide whether a request reads, writes, or erases can therefore issue every request through one access, which probes the tables, returns the key's current value, and then appends the record that the request asks for. [Tianyao: Report the latency of a write without probes against a lookup once the library exposes it.]

Shadowing also hides repeated lookups of an absent key. Such a lookup finds no record to write back. H2O2RAM then appends an unkeyed dummy record, so a repeated lookup of the key probes the same buckets again and reveals the repetition [25]. Following earlier work [23], [24], MICRORAMA instead appends a tombstone, exactly as an erasure does. A later access for the key then finds the tombstone first, so the tables already probed for the key receive only dummy probes. The top level eventually discards tombstones (Appendix A).

Incremental merges. When the online stash fills, its records may move up several levels at once. H2O2RAM and MegaBlocks rebuild all of these levels in one step, building a single table from the stash and the tables of the levels [10], [22]. We instead view this rebuild as a cascade of merges of two tables at a time: the stash with the smallest hash table, the result with the next table, and so on. We call each completed table that a level passes to the next level a *carry*. Unlike a rebuild in one step, a cascade produces a usable table after every merge. Skipping ahead, Section 3 uses this to run the merges in the background: accesses continue on the tables built so far, so they do not stall while a large table is rebuilt.

Every merge of a cascade routes records by the hash of the cascade's final table, so the merges together form one butterfly network over the final table's buckets (Figure 3). Comparing to a one-step rebuild, our incremental merges essentially perform the same work, only in a different order. The earlier merges have already routed the carry, so a merge's *preparation* routes only its level's table, the *older table*, and the merge's last stage then combines the two tables.

Duplicate keys. Merging two tables at a time also makes deduplication easy to analyze. Each input table holds at most one copy of a key, so a merge need only keep the newer copy when a key occurs in both. Goldreich and Ostrovsky remove such duplicates by sorting [5]. Our merges avoid this sort: because a record's routing tag is a hash of its key, both copies reach the same merge-split in the last stage of their

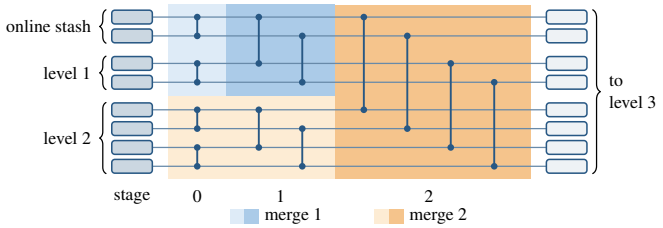


Figure 3: A cascade of merges in a binary construction, viewed as one butterfly network over 8 buckets. Each color marks the merge-splits of one merge: merge 1 combines the stash with level 1's table, and merge 2 adds the table of level 2. In the lighter part of each merge, every table is routed alone. The darker last stage combines two tables, and only its merge-splits compare keys.

merge. Only that stage, drawn darker in Figure 3, compares keys. A cascade that ends in a table of m buckets thus runs $(m/2)\log_2 m$ merge-splits, of which fewer than m compare keys.

The comparison must not reveal which keys match. The merge therefore compares every key in a bucket of the older table with every key in the paired bucket of the newer table, and marks each older record whose key matches as a dummy in the occupancy mask. Two buckets of 64 keys need 4096 key comparisons. A 512-bit register holds 16 keys of 4 bytes or 8 keys of 8 bytes, so with AVX-512F alone, comparing two buckets takes about 580 instructions for 4-byte keys and 1100 for 8-byte keys. We optimize for AMD Zen 5, which supports the VP2INTERSECT extension. Its instructions compare every 4-byte or 8-byte lane of one register with every lane of another, so comparing two buckets takes only 16 and 64 instructions, respectively. On a Zen 5 processor, this makes a merge with 4-byte keys 1.7 times faster (Section 5.4).

A key longer than 8 bytes does not fit in one lane, so MICRORAMA stores it as its 16-byte hash under a secret key and compares the hashes in four 4-byte pieces. A merge then drops a record when every piece of its hash matches the same piece of some hash in the other bucket. The four matches may come from different hashes, in which case the record is dropped by mistake. Nonetheless, because the hashes are secret and random, each piece matches by chance with probability at most $64 \cdot 2^{-32}$, so such a mistake occurs with probability about 2^{-104} per record in each bucket comparison.

2.4. Resolving Remaining Bottlenecks

Byte-sliced merge-splits leave a rebuild with little computation, so three other problems now limit hierarchical ORAM. First, lookups become slow relative to rebuilds. A rebuild processes whole tables in bulk. Each merge-split takes a few register permutations per byte position, and the merge-splits of one stage can run in parallel. A lookup instead probes one bucket in each table, and whether each probe is real or dummy depends on the results of the earlier probes. Vector units and additional cores do little to shorten this chain of dependent memory accesses, so every table on the lookup path adds to the latency of every access. Second, memory traffic now limits the speed of a rebuild, because each stage of the network reads and writes every bucket of the table but does little work on each byte. Third, the 64-slot buckets that make merge-splits fast overflow routinely, so a rebuild must handle the records that do not fit.

Fewer lookup probes with a larger branching factor. To balance the rebuild and lookup costs, we deliberately give rebuilds more work so that each lookup probes fewer tables. We increase the *branching factor* B, the capacity ratio between successive levels, so that one level spans the capacity range of $\log B$ binary levels. MegaBlocks [22] makes the same trade in external memory, where rebuilds are cheap for a different reason: a rebuild processes many records with good locality, whereas each lookup makes random transfers from backing storage. In MICRORAMA, rebuilds are cheap because merge-splits need little computation, so the trade is worthwhile even when all tables reside in memory.

A larger branching factor also reduces storage overhead, because the levels below the largest one together have about $1/(B-1)$ of its capacity: as much as the largest level when $B = 2$, but one seventh of it when $B = 8$.

With $B > 2$, a level receives B carries before it passes a table up. As in MegaBlocks [22], each level keeps one table, so a lookup still probes one table per level. A level adopts its first carry as its *partial table* and merges each later carry into it. Every carry into level ℓ occupies U_ℓ buckets, so a partial table of j carries has jU_ℓ buckets, which need not be a power of two (Appendix B). The merge of the B-th carry ends the level's *cycle* and produces an *outgoing table* of BU_ℓ buckets, which leaves the level as one carry into the next level. We number the hashed levels $1, \ldots, L$ from the smallest, the *bottom*, to the largest, the *top*. The top never passes a table up. It instead merges every carry with its table into a replacement of the same $T = BU_L$ buckets, which hold $N = 32T$ live keys at the nominal load of Section 2.2.

Cache and register locality. The butterfly network has a locality that prior work exploits [19], [33]: the merge-splits of k consecutive stages combine buckets only within groups of 2^k. Routing each group through these stages while it remains in cache reduces passes through memory by about k times. We use a cache-oblivious recursive order [19], [40], which needs no cache-size parameter and exploits this locality at each cache level.

The 64-slot buckets chosen for register permutations also improve this reuse. At the same record size, a cache holds eight times as many 64-slot buckets as 512-slot buckets, so a group covers three more routing stages before its data must leave the cache.

Registers give the same benefit at the smallest scale. We fuse two consecutive stages over four buckets, so each bucket's current byte line stays in a register through both stages. This halves the loads and stores of bucket byte lines while preserving each stage's routing permutations. We measure the combined benefit of fusion and cache-oblivious traversal in the online full version.

Overflow and residual records. At the nominal load, the number of records routed to one 64-slot bucket is approximately Poisson with mean 32, which exceeds 64 with probability about 2^{-22}. A rebuild of a table of 2^{20} buckets writes every bucket at each of its 20 stages, so about four buckets overflow on average.

We cap each bucket of the network at its 64 slots and send the records that do not fit, which we call *residuals*, to a stash shared by many merge-splits. We adapt this idea

from NanoGRAM [21], which caps the buckets of its ORAM and collects all overflow in one shared pile. The overflows of independent buckets rarely occur together, so a shared stash needs little capacity. One merge-split alone needs a stash of 32 slots to keep the probability that the stash overflows below 2^{-64}, whereas 2^{20} merge-splits can share a stash of 62 slots. The loads of consecutive stages, however, are positively correlated, and a tight bound on their total overflow remains an open problem. NanoGRAM, whose pile collects overflow from all levels, sizes the pile by empirical simulation. We instead give each stage its own stash and combine the stages with a union bound (Appendix A).

The union bound is loose, but lookups never scan the stage stashes, which would add 20 stashes to every lookup in a table of 2^{20} buckets. Most residuals overflowed at an intermediate stage, although their destination bucket has room, so a final *sweep* over the output table gives them a second chance (Figure 4). Writing a residual directly into its destination would reveal that bucket. The merge instead obliviously partitions the residuals by destination into one *range bucket* for each range of consecutive output buckets, and the sweep reads the range bucket at every output bucket in its range, whether or not any residual belongs there. The few records that still do not fit form the new table's *residual tier*, which a lookup scans in full whenever it probes the table. As a result, the tier is as small as any placement of the table's keys into 64-slot buckets can make it, and we show one or two buckets suffice for each tier in our default configuration (Appendix A). The tier is offered to the sweep again at the table's next merge, where a different hash is used.

Duplicate keys and the stash bound. The stash bound of each stage requires the routed records to have distinct keys. Because our hash tables are read-only, however, an access leaves the older copy of its key in place, so the two tables of a merge can hold copies of the same key. At first glance, this seems harmless: both copies carry the same routing tag, so they meet in the merge's last stage, whose merge-splits keep only the newer copy (Figure 3). The problem lies in the earlier stages, where each table is routed through its own part of the network and the two parts share each stage's stash. The two copies of a key follow corresponding routes in the two parts. In the extreme case, repeated accesses leave the two tables with exactly the same keys, and every overflow in one part recurs in the other.

A merge's preparation therefore routes the older table through every stage, including the last, and the carry joins only in the sweep (Figure 4). The two tables thus meet only in the sweep, which removes duplicates as in Section 2.3.

Fixed capacities. How many records overflow must not show in a rebuild's memory trace. Every stash, range bucket, and tier therefore has a public capacity that depends only on the table size, and every merge-split visits its prescribed stash storage even when no record overflows. With fixed capacities, the scheme fails only if a record arrives at a stash, range bucket, or tier that is already full. We implement a checker that bounds this probability for a given configuration. For the default configuration with 2^{27} records, the probability that some capacity is exceeded within 2^{32} accesses is about $2^{-69.7}$, below our target of 2^{-64}. We also prove in Lean that a model of MicrORAMa exceeds no capacity within these accesses except with probability at most 2^{-69} (Appendix C). [Tianyao: Should the threat model state that the request sequence is fixed independently of the hash functions? The checker assumes this non-adaptive model, and an adaptive client needs a reduction that is not yet proved.]

3. Deamortized Rebuilding

Without deamortization, an access of the hierarchical ORAM of Section 2 waits for any rebuild it triggers, and requests that arrive meanwhile queue behind it. We therefore run rebuilds in the background while accesses continue, balancing latency, throughput, and storage overhead.

3.1. Background Rebuilding and Pre-routing

Background rebuilding. Following Ostrovsky and Shoup [11], each level answers queries from *published* tables while a background worker builds replacements. The worker merges the partial table with each carry from below (Section 2.3), reading published inputs and writing to separate storage. Publishing an outgoing table passes it to the next level as a carry.

With enough cores, each level can rebuild on its own core. When there are fewer cores than levels, the workers of consecutive levels share a thread. The thread runs their merges in short pieces, in earliest-deadline-first order, so that a long merge at a large level does not block the rebuild of a small level. [Tianyao: Report the longest request stall with this schedule against running each merge whole. Dev host, 32M records, one thread: 0.66 ms against 0.68–0.90 s.]

Background rebuilding adds no rebuild work, but costs extra lookup probes and storage. Until a merge publishes its output, every lookup must search its carry as a separate table, and the merge's inputs and output occupy memory at the same time. To reduce the extra probes, we want each merge to publish its output soon after its carry arrives.

Pre-routing. To publish sooner, a merge starts its preparation early. Preparation does not read the carry (Section 2.4), and the public rebuild schedule determines which hash the output uses (Appendix B), so the merge starts preparation as soon as the partial table is published, before its carry arrives. With this *pre-routing*, only residual partitioning and the sweep remain when the carry arrives. The top uses the same procedure.

Pre-routing also keeps the required rebuild speed close to the average. In an idealized model where preparation dominates the cost of a merge (Appendix B), each non-top level at $B = 8$ must rebuild at $4.5\times$ the access rate, only about 3% faster than its average rate of $4.375\times$. [Tianyao: Determine what fraction of rebuild work must be available for pre-routing for deamortization to require no additional rebuild speed above the same-B average, and whether our construction can attain it.]

3.2. Storage Reuse

Storage cost of deamortization. The remaining cost of deamortization is storage. A merge cannot overwrite its inputs while accesses still read them, so old and new tables coexist at every level. With half-full hash tables, prior deamortization schemes therefore reserve at least $10\times$ the space of the raw

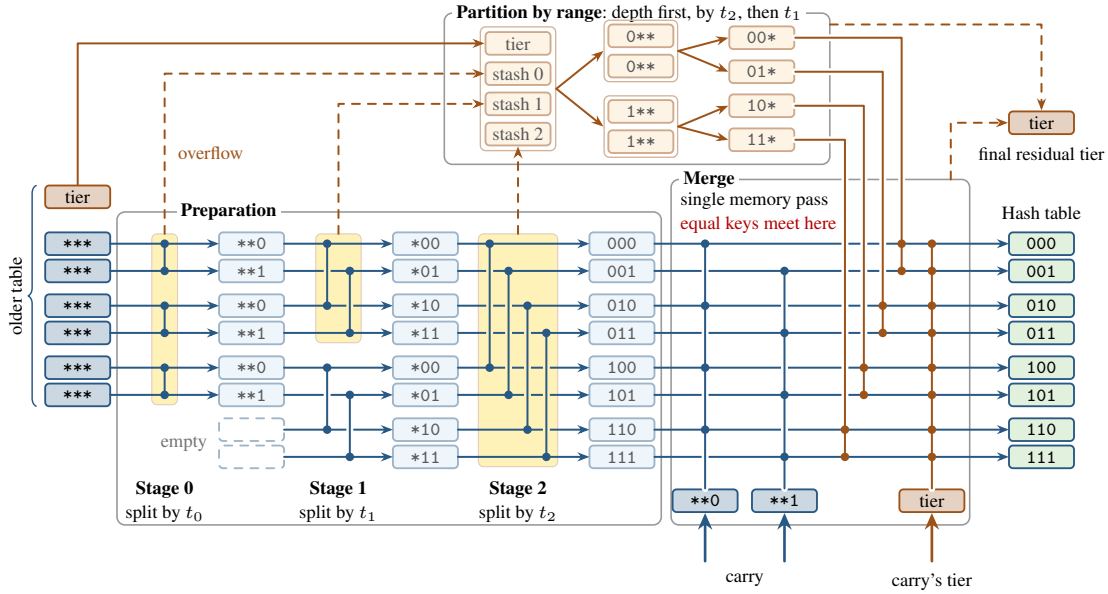


Figure 4: A merge adds a two-bucket carry to a six-bucket older table. Each small box is one 64-slot bucket. Labels give fixed routing-tag bits, with $*$ marking an unfixed bit. Intermediate buckets are lighter, and shaded merge-splits can overflow. Preparation routes only the older table, collecting overflow in a separate stash per stage. The partitioning tree divides the older table's tier and these stashes into range buckets. At each output bucket, a single sweep offers the pre-routed carry, range residuals, and the carry's tier in that order. Unplaced records form the final residual tier.

data [11], [12], [13] (Table 1). We cannot simply reserve less, because a level without a free region for its next merge makes the carries from below wait, and the wait propagates down to the online stash and stalls foreground accesses.

Storage layout. In our design, a level below the top can hold three tables at once. Its previous outgoing table stays queryable until the next level absorbs it, and its partial table stays queryable while pre-routing builds the replacement in separate storage. Three full-size regions are not necessary, however, because a partial table holds at most $B-1$ carries. Each level therefore reserves two regions of BU_ℓ buckets and one region of $(B-1)U_\ell$ buckets (Figure 5). The partial table and its replacement alternate between the smaller region and one full-size region, timed so that the last merge of the cycle writes into the full-size region. The full table is then already where the next outgoing table belongs, so it needs no relocation copy. Once the next level retires the previous outgoing table P, its region pairs with the smaller one for the next cycle.

Storage at the top. The top matters most for storage, because it holds a fraction $(B-1)/B$ of the capacity: half at $B=2$ and seven eighths at $B=8$. Its published table must stay queryable while the replacement is built, so the top needs at least two regions of T buckets. MICRORAMA needs no more, because the butterfly network rebuilds a table in place: preparation routes the published table into the second region, and the sweep then inserts the carry there.

Storage bound. The three regions of a non-top level hold $3-1/B$ times its full table, and the two regions of the top hold twice its table. With half-full hash tables, MICRORAMA with $B=2$, 4, and 8 therefore uses about $9\times$, $5.8\times$, and $4.8\times$ the space of the raw data, respectively. These numbers ignore metadata. We compare this cost with prior schemes in Section 3.5.

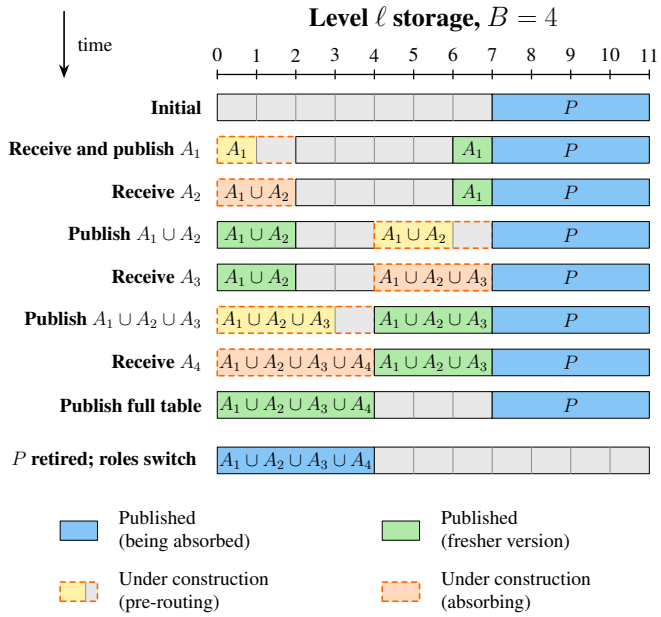


Figure 5: Logical storage of a non-top level ℓ with $B=4$, with time running down the rows. Each unit holds U_ℓ buckets, and the three regions hold $4U_\ell$, $3U_\ell$, and $4U_\ell$ buckets, or $11U_\ell$ in total. Carries $A_1, \ldots, A_4$ arrive in order, and a union denotes a merge that keeps the newest copy of each key. Green tables and the older blue table P stay queryable while the dashed outputs are built. A gray region is reused only after the readers of its previous table finish.

Scheme	Table type	Storage (slots)	Rebuild speed required	Tables per lookup	
				Average	Worst case
Amortized baseline ($B = 2$, load factor 0.5)	Bucket hash + stash	$4N$	$1\times$ (avg.)	$0.5 \log N$	$\log N$
Ostrovsky–Shoup [11], Sect. 5	Bucket hash	$\geq 12N$	$6\times$	$0.83 \log N$	$\log N$
Goodrich et al. [12]	Cuckoo hash	$\geq 11N$	$2\times$	$1.5 \log N$	$2 \log N$
PD-ORAM [13]	Hash + BF	$\geq 10N$	$\geq 2\times$	$1.5 \log N$	$2 \log N$
MicroRAMa, general B	Bucket hash + stash	$\left(4 + \frac{6B-2}{B(B-1)}\right)N$	—	—	$2 \log_B N$
MicroRAMa, $B = 2$		$9N$	$1\times$	$1.5 \log N$	$2 \log N$
MicroRAMa, $B = 4$		$5.83N$	$2.25\times$	$0.92 \log N$	$\log N$
MicroRAMa, $B = 8$		$4.82N$	$4.5\times$	$0.59 \log N$	$0.67 \log N$

TABLE 1: Costs of deamortization for N live records, predicted by the idealized model of Appendix B. The entries compare schedules, not the end-to-end performance of implementations. Storage counts the record slots of each scheme's tables, rebuild arrays, and history lists, with every hash table at load factor 0.5, so dividing by $4N$ gives the overhead over an amortized binary hierarchical ORAM with the same table type. Rebuild speed is how fast each level below the top must rebuild records so that accesses never stall, as a multiple of the access rate. The amortized baseline pauses accesses instead, so its entry is its average rate. The Ostrovsky–Shoup speed charges the whole rebuild after the second carry arrives. Tables per lookup count the separately queryable tables. Entries marked $\geq$ are lower bounds, log has base two, and BF denotes a Bloom filter.

3.3. Foreground Accesses

With rebuilds in the background, the access itself becomes the bottleneck. It is a chain of dependent memory loads (Section 2.4), because which bucket to load next is known only after the current bucket's keys have been compared. To keep other computation off this chain, a single thread computes the hashes of all its probes before the chain starts and extracts each table's value while the next bucket is being fetched.

Additional threads help most when an application submits independent requests in batches. We divide each batch among several threads, and each thread processes up to 64 requests together, one table at a time. Batching speeds up lookups even on a single thread, because the bucket loads of all requests that a thread processes together are in flight at the same time. On Intel TDX with 48-byte values, a batch of 64 requests on one thread raises the throughput by $1.20\times$ with 2^{20} records and by $1.31\times$ with 2^{25} records.

A batch can contain the same key twice, and probing the same buckets twice for one key would reveal the repetition. We therefore find repeated keys with AVX-512 comparisons across the whole batch, and only the first occurrence of each key probes for real. Each later occurrence issues dummy probes and takes its result from the occurrence before it, so the requests of a batch take effect in batch order. Snoopy's load balancer deduplicates its batches in a similar way [41], whereas SONIC requires the application to prevent concurrent requests for the same block [32].

3.4. Correctness and Security

Background merges must keep both access guarantees of Section 2: a lookup returns the newest value of its key, and a key never receives two real probes under the same hash, so the probes do not reveal whether a key is accessed again.

Freshness order. Without background merges, smaller levels hold newer records, so searching the levels in order finds the newest copy first. While merges run, however, a level can hold several queryable tables, so we order the tables by when their records were written. Each table holds the records of a contiguous range of accesses, and a lookup searches the tables from the newest range to the oldest. A merge combines two tables with adjacent ranges, and its output takes the place of its inputs in this order. The merge also uses this order to decide which of two copies of a key is newer, so lookups always return the newest value.

Thread safety. Each access searches a snapshot of published tables, its *probe plan*. A merge atomically replaces its inputs with its output in later probe plans, so an access sees either all inputs or the complete replacement. Input regions are reused only after earlier readers finish. If workers fall behind and a level exhausts its free regions, foreground accesses wait for a merge to complete and earlier readers to release its inputs.

Security. The adversary now also sees when each merge runs and publishes its output, but this rebuild schedule depends only on when requests arrive, not on their content or the stored data.

For foreground accesses, one notable difference caused by deamortization is that a single hash serves several queryable tables in turn. The merges of a cascade share one hash (Section 2.3), and a carry stays queryable until its merge atomically replaces it with an output of the same hash. A key nonetheless receives at most one real probe under this hash. After the real probe, the access writes the key's record or a tombstone to the online stash, so later lookups find it first and send only dummy probes to every table with the hash.

The rest of the security analysis largely follows from prior work [5], [10] and the discussion in Section 2. In Appendix C, we describe how we machine-check in Lean the capacity bound of Section 2.4 and the correctness and security of our design. [Tianyao: TODO: Appendix C formalizes the hash-table view, including the publication trace. Formalize timing and the scheduling of the workers, and verify the compiled masked kernels under the selected enclave model. Add a cross-reference to the residual analysis giving capacities, request horizon, and the combined construction-failure bound.]

3.5. Comparison with Prior Deamortization Schemes

Table 1 compares MICRORAMA with the deamortization of Ostrovsky and Shoup [11] and of two later schemes [12], [13]. Its entries come from an idealized model of each scheme's rebuild schedule (Appendix B). The model counts the record slots that each scheme reserves and the tables that a lookup searches. It also finds the slowest rebuild speed, as a multiple of the access rate, at which no access ever waits for a rebuild. For MICRORAMA, the model assumes that pre-routing completes all of a merge's work except the carry's share before the carry arrives. [Tianyao: TODO: replace the general-B table dashes with compact forms of Eqs. (1) and (2) for rebuild speed and average table count.]

While MICRORAMA shares the high-level idea of Ostrovsky and Shoup [11, Section 5], it uses storage in two different ways. First, their rebuild needs a work array twice the size of the table, whereas our butterfly network rebuilds a table in place (Section 3.2). Second, Ostrovsky and Shoup keep only two versions per level, so a level must wait for the next level to absorb its older version before it can start its next table, which is also observed by Asharov et al. [42]. Our analysis shows that with $B = 2$, each level must then rebuild at $6\times$ the access rate to avoid stalls (Appendix B). In MICRORAMA, every level except the top instead has a third, smaller region, so it can start its next table while the next level is still absorbing the older one. Together with pre-routing, this keeps MICRORAMA with $B = 2$ free of stalls as long as each level rebuilds records as fast as accesses insert them.

We instead use our margin in rebuild speed to raise the branching factor. With $B = 8$, each level spans the capacity range of three binary levels, so a lookup searches fewer levels. The levels below the top, which need three regions each, also shrink to one seventh of the top's capacity.

4. Tree-based ORAM for Large Entries

For large entries, MICRORAMA stores the values of the entries in a Path ORAM [14] and uses our hierarchical OMAP as its position map. Path ORAM keeps each value as a *block*, either in a tree [43] whose nodes are buckets of Z slots or in a small stash. The position map assigns each block a uniformly random leaf, and a block outside the stash always lies on the path from the root to its leaf. An access reads the whole path of the requested block, gives the block a fresh random leaf, and writes the path back. Because a block's leaf is redrawn at every access, the paths that accesses read are independent of the requested keys. For an absent key, the access instead reads the path of a random leaf. The write-back, called the *eviction*, fills the buckets of the path from the leaf upward, so that each block moves as deep as its own path allows. Blocks that fit nowhere stay in the stash, and the construction fails if the stash overflows.

4.1. Eviction by Register Permutation

Permuting the working set. Inside a TEE, the eviction must hide where each block goes, so the locations it reads and writes must not depend on any block's destination. An eviction rearranges its *working set*, the slots of the stash and the path, by a permutation that the blocks' secret leaves determine. As in our merge-split (Section 2.2), a permutation inside vector registers can depend on these leaves without revealing them. `VPERMT2B` selects each of its 64 output bytes from the 128 bytes of two registers, so we limit the working set to 128 slots: a stash of 64 slots and a path of at most 64 slots. In a byte-sliced layout, the same byte of all 128 slots then fills two registers, and two `VPERMT2B` instructions, one producing the stash and one producing the path, move that byte of every block to its destination. Evicting w-byte blocks thus takes $2w$ instructions, whichever permutation the leaves select.

The stash is stored byte-sliced, and a few permutations transpose loaded paths into byte lines and back.

Computing the permutation. We reserve stash slot 0 for the requested block, which the access reads and updates there. The permutation depends only on block identifiers and leaves, so we apply Path ORAM's greedy rule to this metadata in registers. From the leaf upward, a 128-bit mask marks unplaced slots whose paths pass through each bucket, and branchless bit operations select up to Z of them. Any remaining blocks go to the 63 unreserved stash slots. The resulting index vectors and masks form the eviction's *plan*. The layout and selection details are in the online full version. Every eviction loads and stores the whole working set at addresses that depend only on the public leaf, and its placement decisions stay in registers, under the timing assumption of Section 2.2. [Tianyao: Check BLSI and BZHI against Intel's DOIT list, and add any that it omits to the timing measurement of Section 2.2.]

Comparison with prior evictions. Most of our saving over prior TEE evictions comes from moving the payload. Oblix moves each block into place with a linear scan of oblivious swaps over the path [7], so it passes over the payload once per block. EnigMap moves blocks with three bitonic sorts over the path and the stash [8], each making at least 28 passes of compare-exchanges over the blocks. Our permutations pass over the payload once, at two instructions per byte position. On the same tree and working set, one eviction of 1 KiB blocks takes 2.5 μs with our permutations, 98 μs with linear scans, and 140 μs with EnigMap's eviction (Section 5.3).

Computing the plan is also faster. Oblix computes the same greedy placement, but it scans the path once for each block, whereas our masks process all 128 slots together at a few instructions per bucket. A bitonic sort is slower even when it moves only metadata: on our development machine, sorting the 128 slots by the deepest bucket each block may occupy takes 55 ns, about twice the 28 ns of the placement loop that it would replace. [Tianyao: Remeasure both on the evaluation machine with four-slot buckets and branching factor four, whose paths are about half as long as the 19-bucket path measured.]

Two evictions per access. Path ORAM evicts only the path that it reads, so a block in the stash descends only as far as that path shares the block's own path, usually just the top levels. While the tree is first filled, no block is read again, so on the tree of Section 4.3 the top levels saturate and the stash grows to hundreds of blocks, far beyond our 63 slots. After each access, we therefore evict a second path, chosen in reverse-lexicographic order [44], which covers every bucket on a fixed public schedule.

Path ORAM's eviction rather than Circuit ORAM's. Cir-

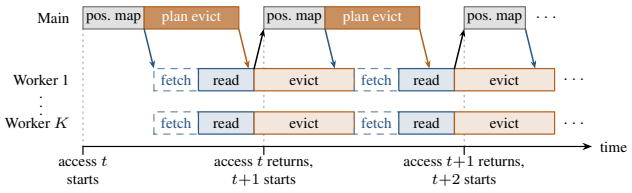


Figure 6: Two consecutive accesses with K payload workers. Blue marks work on the access path, which the access reads and evicts, and orange marks work on the deterministic eviction path. Downward arrows mark when the main thread publishes a path's leaf and plan, and the main thread also applies each plan to the metadata bytes. The upward arrow marks when every worker's part of the requested payload is complete, which returns the access and starts the next one. The workers are the bottleneck, so a worker starts the next fetch only after it finishes evicting, and gaps in the main thread's row are idle time. Lengths are schematic.

cuit ORAM [15] lets each bucket pass at most one block deeper per eviction. This restriction gives a nearly optimal eviction circuit, but places fewer blocks. With register permutations, it saves nothing: every permutation of the working set costs the same $2w$ instructions. We therefore keep Path ORAM's eviction, which places as many blocks on the path as any valid placement can. Simulations show that Circuit ORAM needs a third eviction per access to approach our stash tail (the online full version).

4.2. Parallel Eviction

An eviction's payload work grows with block width, but every byte uses the same plan. We therefore split each block's payload into contiguous ranges, one per worker. Each worker keeps its range in a separate tree and stash, so workers write disjoint memory. The main thread owns the metadata, looks up the position map, and computes the plans. Workers receive only leaves and plans. Applying the same plan to every byte range preserves the single-threaded placement, and each worker's memory accesses depend only on the public leaves. The public worker count defaults to one per 128 payload bytes, up to four.

An access need not wait for its second eviction. The permutation that evicts the access path also moves the requested block into stash slot 0, so a worker's part of the payload is complete once it has evicted its byte range of the access path. The access then returns, and the deterministic eviction overlaps the next request and its position-map lookup (Figure 6). This schedule removes the deterministic eviction from an access's latency. Sustained throughput still depends on both evictions and is bounded by the slower of the main thread and the workers. Section 5.3 measures the gain from payload workers.

4.3. Bucket Size and Branching Factor

Every working-set permutation costs the same as long as the path fits in 64 slots, so we choose the bucket size and branching factor for locality. Following the higher-degree trees of Gentry et al. [44], we use four-slot buckets and branching factor four, supporting up to 2^{32} blocks. For 2^{18} blocks, a path has 9 four-slot buckets instead of 17 two-slot buckets in a binary tree, reducing random accesses. Larger buckets also support more workers without shortening their sequential reads (the online full version). The cost is storage: to keep the stash tail short, we use 0.375 blocks per slot instead of the binary tree's 0.5, requiring 1.33 times as many slots. [Tianyao: Report the measured speedup over the binary tree with two-slot buckets, on one thread and with four workers.]

4.4. Failure Probability

A stash overflow is fatal because the position map has already changed when the eviction fails. We therefore target a failure probability below 2^{-64} over 2^{32} accesses. Path ORAM proves an exponentially decreasing stash tail only for five-slot buckets in a binary tree that evicts only the access path [14]. No proof covers our tree, so we measure its stash tail.

We simulate up to 7×10^{11} accesses each for 2^{20} and 2^{21} blocks under uniformly random and sequential (round-robin) reads. The worst case is sequential reads of 2^{21} blocks. A straight line fitted to its measured tail predicts that 49 stash slots meet a failure probability of 2^{-96} per access, or 2^{-64} over 2^{32} accesses by a union bound. The measured tail falls more slowly at its deepest points; if it keeps falling at their rate, 54 to 56 slots are needed, and the 63-slot stash still overflows with probability below 2^{-108} per access. These estimates extrapolate tails observed to about 2^{-32}. The simulation methods and tail curves appear in the online full version. [Tianyao: State this empirical basis in the threat model, or prove an analytical bound.]

5. Evaluation

5.1. Methodology

Machines. We run every experiment on two cloud virtual machines with 16 physical cores each:

- **Intel TDX:** a Google Cloud `c4-standard-32` virtual machine with an Intel Xeon 6985P-C (Granite Rapids) processor, a 504 MiB last-level cache, and 117 GiB of memory, running as an Intel TDX trust domain.
- **AMD SEV-SNP:** an Oracle Cloud `VM.Standard.E6.Flex` virtual machine with an AMD EPYC 9J45 (Zen 5) processor, whose cores form two core complexes of eight, each with a 32 MiB last-level cache, and 128 GB of memory, running as an AMD SEV-SNP guest. Unlike the Intel machine, it supports VP2INTERSECT. We check the attestation report of every run on it against AMD's key server.

On both machines, we use one hyperthread per core. One core runs the benchmark harness, so MicroRAMA uses the other 15; each baseline may use up to all 16. We additionally launch two otherwise identical virtual machines without TDX and SEV-SNP to measure the TEE overhead (Section 5.4). All code is compiled at `-O3 -march=native`, with GCC 15.2 except for SONIC, whose build requires Clang.

Workload and metrics. We compare the systems at three record sizes, each matching a use case (Table 2). With 4-byte keys and 8-byte values, the map serves as an ORAM of 8-byte words, the standard interface for general-purpose oblivious

computation. With 8-byte keys and 48-byte values, it holds the records of Signal's contact discovery service, which maps each 8-byte phone number to three 16-byte identifiers [45]. Records with 32-byte keys and 1 KiB values approximate larger objects. For example, the nodes of Ethereum's Merkle Patricia trie are stored under their 32-byte hashes [46] and limited to 1 KiB by the Portal Network [47], and the embedding vector that a large language model looks up for each token usually exceeds 1 KiB [48]. Unless stated otherwise, the map holds N records with dense 4-byte keys and 8-byte values, and requests are 95% lookups and 5% updates of keys drawn uniformly at random, so every lookup finds its key. Throughput is the number of completed requests per second when the caller issues the next request as soon as the previous one returns. After at least N warm-up requests, each run measures at least N requests, spanning at least two top-level merges. For latency, requests arrive at fixed intervals, and response time includes waiting from the scheduled arrival. We report one run per configuration. On five boots of the Intel machine, the throughput at 2^{27} records varied by up to 18%, and on four boots of the AMD machine by at most 2.7%. [Tianyao: Add a second trial of the headline rows if time allows, and state the spread.]

Baselines. We run the three baselines as follows:

- **H2O2RAM** [10]: we patch its released code [25] to fix two record-loss bugs and two leaks: a secret-dependent branch and fixed dummy-probe locations in bucket and cuckoo tables that reveal the hit level. We report the patched build and identify comparisons with the released build. Before each measurement, H2O2RAM times candidate hash tables to select its configuration. This preparation needs far more memory than the map: with 2^{27} records it takes 12.1 h and 115 GiB. We therefore prepare the maps of Table 2 on an otherwise identical AMD virtual machine with 256 GB of memory, and the Intel runs reuse the configurations it selects.
- **SONIC** [32]: the ZingORAM scheme of its artifact, an ORAM that serves dense keys $0, \ldots, N-1$. We use it at every record size because SONIC's oblivious map is too slow for our experiments. SONIC's measurements keep the position map as a plain array outside the ORAM, and so does our first configuration, which we call SONIC without a position map. Because a plain array leaks each request's position, our second configuration builds a recursive position map from ZingORAM trees, which makes SONIC oblivious; we call it simply SONIC. SONIC crashed in our long runs because its threads share one pseudorandom generator, so we give each thread its own. SONIC's paper describes deduplicating the requests of a batch, but its artifact does not implement it, so we assume that the requests of a batch are already distinct.
- **Non-oblivious hash table**: a single-threaded `std::unordered_map` with its default maximum load factor of one. It hashes each key's bytes with libstdc++'s string hash, MurmurHash2. Given a batch, it runs the lookups back to back.

Both oblivious baselines use AVX-512 instructions: they split large entries into 64-byte chunks and conditionally copy or swap each chunk in constant time. SONIC's build leaves these instructions off by default, and we enable them.

5.2. Hierarchical ORAM for Small Entries

Throughput without batching. With small entries, MicroRAMa serves one request at a time 12.8 to 15.4× and 9.7 to 14.1× faster than H2O2RAM on the Intel and AMD machines, respectively, up to 2^{27} records (Figure 7). Most of the gap comes from rebuilding. H2O2RAM rebuilds its tables with bitonic sort, which takes 56 to 258× as long as our network on the Intel machine (Section 5.4), and the access that triggers a rebuild runs it before the next request. [Tianyao: Confirm with a profile of H2O2RAM that rebuilds take most of its time per access.] H2O2RAM's released build, which keeps the secret-dependent branch, is at most 26% faster than the patched build, so the gap is not the cost of our security fixes. With 48-byte values and 2^{25} records, MicroRAMa remains 10× faster than H2O2RAM on both machines (Table 2).

SONIC fares no better with one request at a time: at 2^{21} records, MicroRAMa is 13 to 14× faster than SONIC without a position map and 41× faster than SONIC on both machines. At 2^{27} records, MicroRAMa is 46 and 42× faster than SONIC on the Intel and AMD machines, respectively, and still 22 and 17× faster than SONIC with batches.

For large maps, MicroRAMa comes close to the non-oblivious hash table, because both are bound by memory latency. At 2^{27} records, MicroRAMa reaches 70% of the hash table's throughput on the Intel machine and is 1.18× faster than it on AMD. Although a lookup probes 14.0 tables on average (Table 3), the small levels stay in the last-level cache, and only the probes of the largest ones go to memory, one after another. [Tianyao: Confirm with cache-miss counters that only the largest levels' probes miss the last-level cache; the regions of levels 1–5 total about 1/49 of the top's.]

Throughput with batching. Batching overlaps the bucket loads of up to 64 requests per thread (Section 3.3), and we divide each batch of 64 requests among eight threads. At 2^{27} records, batches raise MicroRAMa's throughput by 3.2× on the Intel machine and 6.5× on AMD. On the Intel machine, MicroRAMa then reaches 65% of the throughput of the single-threaded non-oblivious hash table given the same batches, and on AMD, 2.4× its throughput. SONIC is designed for batches, yet with the same batch size, MicroRAMa is 11 to 20× faster than SONIC without a position map and 40 to 82× faster than SONIC up to 2^{21} records on the two machines. At 2^{27} records, it is 70 and 114× faster than SONIC on the Intel and AMD machines, respectively. Additional size-sweep and batch-size results appear in the online full version.

Response time. Because MicroRAMa rebuilds in the background, a request waits only for its own probes until the load approaches MicroRAMa's throughput. At half of its throughput, MicroRAMa therefore answers one request at a time within 6 μs even at the 99th percentile, like the non-oblivious hash table (Table 2). H2O2RAM also answers half of its requests within 9 μs, but it rebuilds its largest level inside one access, for about 117 s on the Intel machine and 90 s on AMD, and the requests that arrive meanwhile queue behind it. With 2^{27} records, these waits make its average response time more than a million times ours. With batches at this size, SONIC's response time is about 80 and 120 times ours

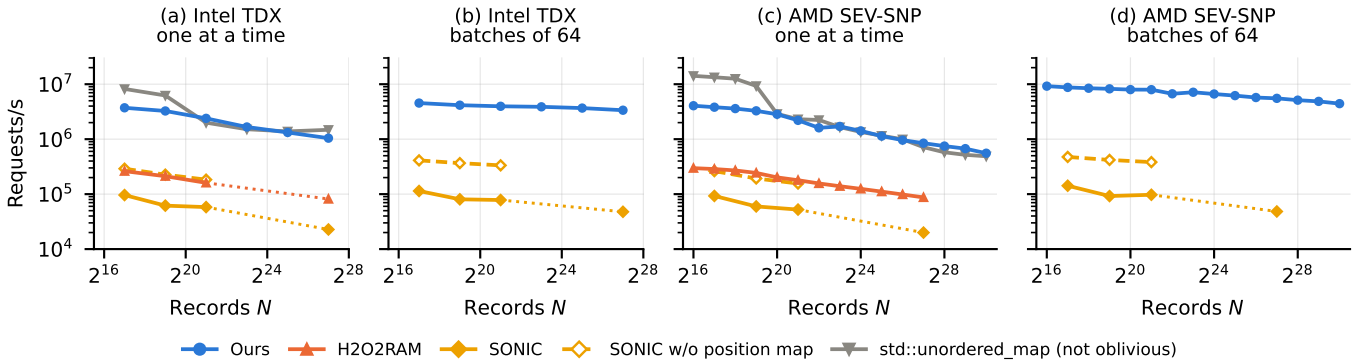


Figure 7: Throughput with 4-byte keys and 8-byte values, by the number of records N, on Intel TDX (a, b) and AMD SEV-SNP (c, d). With batches, MicroRAMA divides each batch of 64 requests among eight threads, and SONIC uses eight access and eight eviction threads. H2O2RAM has no batch interface. SONIC, and H2O2RAM on Intel TDX, were measured up to 2^{21} records and at 2^{27}, where SONIC's batches hold 8 requests; at 2^{21}, batches of 8 to 512 requests give it the same throughput within 1%.

on the Intel and AMD machines, respectively, because it runs each batch of 64 as eight batches of 8 in turn.

Memory. MicroRAMA also uses less memory than both oblivious baselines: at 2^{27} records, 2.2× less than H2O2RAM and 2.8× less than SONIC (Table 2). Most of this memory is the library's allocation, 6.5× the bytes of keys and values, and 99% of that is in the main tables. These tables hold 4.82 slots per record, and each 16-byte slot stores a 12-byte record together with its metadata and routing tag, which gives a factor of 6.4. [Tianyao: The 4.82 slots and the 16-byte slots are inferred from the 9,875 MiB of main tables; confirm the slot width in the code.]

[Tianyao: The Intel H2O2RAM cells reuse the configurations selected by its preparations on AMD, which took 115 GiB, 166 GiB, and more than 186 GiB, against the Intel virtual machine's 117 GiB. The new AMD SONIC cells ran while that machine's effective last-level cache was reduced, which slowed SONIC's control run by 14% one at a time and 7% with batches. SONIC's throughput with batches at 2^{27} is still with batches of 8.]

5.3. Hybrid with Tree-based ORAM

Hierarchical versus tree-based ORAM. For large values, the tree construction serves requests faster than the hierarchical ORAM and needs less memory. It overtakes the hierarchical ORAM between 64 and 128 bytes on both machines (Figure 8), so MicroRAMA selects the tree from 128-byte values on. The two constructions pay for a value in different ways. A hierarchical lookup loads the value bytes of all 64 slots of a bucket in every table it probes, or $64w$ bytes per table for a w-byte value. A tree access reads and writes two paths of about 40 slots each, or about $160w$ bytes, but it first pays a position-map lookup in the hierarchical ORAM. Small values make this fixed cost dominate, and large values make the per-byte traffic dominate, where the tree moves several times fewer bytes. With 1 KiB values, the tree therefore serves 5.4× as many requests as the hierarchical ORAM on the Intel machine (4.7× on AMD). It also uses 44% less memory, because it stores each value once instead of in levels that hold several versions of every table. [Tianyao: The traffic estimates are back-of-the-envelope (two paths of 10 four-slot buckets; about a dozen tables per lookup at 2^{20}); check them against the probe-plan and path counts at 2^{20} before keeping them. Decide the threshold: on AMD the curves cross at 67–73 B, so with the 128-byte threshold, 96-byte values run on the hierarchical ORAM, 6–13% slower than on the tree. The Intel tree rows predate the 64-byte payload grain; rerun them.]

Comparison with H2O2RAM. Our hierarchical ORAM alone serves 8 to 12× as many requests as H2O2RAM on the Intel machine at every value size up to 512 bytes, and 10 to 18× as many on the AMD machine up to 1 KiB (Figure 8). The tree, which keeps large values out of the hash tables and their rebuilds, widens the gap as values grow. With 2^{21} records of 1 KiB values, MicroRAMA serves 70 and 72× as many requests as H2O2RAM on the Intel and AMD machines, respectively, and uses 6.0× less memory (Table 2). [Tianyao: H2O2RAM's 2 KiB trial failed the assertion `matches == left_cnt` in its oblivious bipartite matching after its preparation finished (5.6 h, 206 GiB). The preparation is cached, so a retry needs only the trial (about 30 minutes).]

Comparison with SONIC. With 2^{21} records of 1 KiB values, MicroRAMA serves 8.8 and 12.8× as many requests as SONIC on the Intel and AMD machines, respectively, and uses 1.6× less memory (Table 2). MicroRAMA remains 5.0 and 8.9× faster than SONIC without a position map (Figure 8), although MicroRAMA's measurement includes the oblivious position-map lookup of every access. This lookup takes only 8 to 19% of a tree access on the Intel machine, whereas making SONIC's position map oblivious slows SONIC by 1.64× (1.61× on AMD). With batches, for which SONIC is designed, MicroRAMA is still 6.5 and 8.4× faster than SONIC on the Intel and AMD machines, respectively.

Cost of eviction. The register-permutation eviction keeps the tree's payload work small: on the same working set on the Intel machine, it is 34 to 58× faster than EnigMap's eviction and 21 to 43× faster than Oblix's linear scans, from 256-byte to 4 KiB blocks (Section 4.1). Payload workers then divide the remaining work: with 1 KiB values, four workers return

TABLE 2: Throughput, response time, and memory for maps of at least 1 GiB of keys and values. Batches hold 64 requests. Response times are at half of each system's own throughput, when requests, or whole batches, arrive at fixed intervals. Memory is the peak resident memory of the process on Intel TDX, excluding H2O2RAM's preparation, and SONIC's is with batches. SONIC serves dense keys $0, \ldots, N-1$. The 1 KiB columns use the tree hybrid. H2O2RAM has no batch interface. The non-oblivious map is a single-threaded `std::unordered_map`.

	Key / value, N		**4 B / 8 B, 2^{27}**				**8 B / 48 B, 2^{25}**				**32 B / 1 KiB, 2^{21}**			
			Intel TDX		AMD SEV-SNP		Intel TDX		AMD SEV-SNP		Intel TDX		AMD SEV-SNP	
Throughput	Single	Ours	1.04 M		845 K		568 K		598 K		283 K		338 K	
		H2O2	82 K		87 K		57 K		59 K		4.0 K		4.7 K	
		SONIC	23 K		20 K		35 K		26 K		32 K		26 K	
		Non-oblivious	1.48 M		715 K		1.49 M		1.06 M		1.17 M		907 K	
	Batch	Ours	3.36 M		5.53 M		1.45 M		2.11 M		305 K		407 K	
		SONIC	48 K		48 K		60 K		62 K		47 K		48 K	
		Non-oblivious	5.14 M		2.33 M		5.46 M		3.78 M		1.98 M		1.57 M	
Response time			Avg	P99	Avg	P99	Avg	P99	Avg	P99	Avg	P99	Avg	P99
	Single	Ours	0.93 μs	1.4 μs	1.3 μs	2.5 μs	1.8 μs	2.4 μs	1.7 μs	2.8 μs	3.3 μs	5.8 μs	2.8 μs	5.6 μs
		H2O2	4.0 s	90 s	2.2 s	61 s	1.5 s	32 s	1.3 s	28 s	0.73 s	20 s	0.76 s	20 s
		SONIC	97 μs	604 μs	86 μs	575 μs	85 μs	535 μs	81 μs	545 μs	130 μs	758 μs	114 μs	729 μs
		Non-oblivious	0.82 μs	1.3 μs	1.5 μs	3.3 μs	0.98 μs	1.3 μs	1.1 μs	2.5 μs	0.80 μs	1.4 μs	1.2 μs	2.8 μs
	Batch	Ours	16 μs	21 μs	11 μs	15 μs	34 μs	45 μs	35 μs	47 μs	236 μs	268 μs	152 μs	164 μs
		SONIC	1.3 ms	2.7 ms	1.3 ms	2.7 ms	1.1 ms	2.5 ms	1.1 ms	2.3 ms	1.6 ms	3.8 ms	1.5 ms	3.6 ms
		Non-oblivious	12 μs	14 μs	28 μs	34 μs	11 μs	12 μs	19 μs	25 μs	23 μs	26 μs	36 μs	42 μs
Memory		Ours	10.0 GiB				9.3 GiB				6.0 GiB			
		H2O2	22.1 GiB				19.7 GiB				36.2 GiB			
		SONIC	27.7 GiB				15.9 GiB				9.8 GiB			
		Non-oblivious	7.3 GiB				3.0 GiB				2.4 GiB			

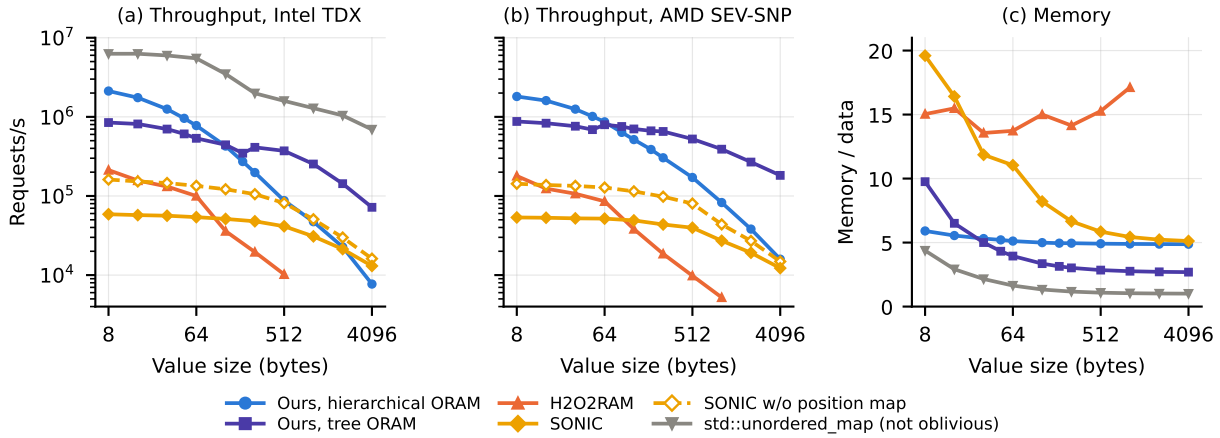


Figure 8: Throughput and allocated memory by value size, with 2^{20} records of 8-byte keys, one request at a time, on Intel TDX (a) and AMD SEV-SNP (b). The hierarchical ORAM uses one fixed configuration at every size. Memory (c) is the library's allocation divided by the bytes of keys and values, which is the same on both machines. H2O2RAM's preparation for 1 KiB values does not fit in the Intel machine's memory, and its run with 2 KiB values on AMD failed an assertion in its code. SONIC without a position map counts only the time of SONIC's own ORAM in the same runs. We ran `std::unordered_map` only on Intel TDX. The memory of the baselines is the process's peak resident memory, excluding H2O2RAM's preparation.

an access 3.4× sooner than one thread on the Intel machine, and six workers 3.1× sooner than one on the AMD machine.

5.4. Discussion of the Results

Cost of rebuilding. Every request writes its record into the smallest level, so every request causes rebuild work. Our network keeps this work small: on the Intel machine, it builds a hash table 56 to 258× faster than H2O2RAM's bitonic sort. Most of the gain comes from the register permutations, which make a merge-split 7 to 8× faster than a bitonic network over the same slots. The cache-oblivious order with fused stages then reduces memory traffic, which makes routing another 1.7 to 2.8× faster, and on the AMD machine, VP2INTERSECT makes a merge 1.7× faster. Further measurements appear in the online full version. [Tianyao: Measure the cache-oblivious

order on AMD.]

Background rebuilding. A fast rebuild alone does not keep requests from waiting. When the request thread runs every merge itself, the access that completes a carry for the top waits 10 s at 2^{27} records on the Intel machine, and every request that arrives meanwhile queues behind it. Background workers remove this pause, and they also raise throughput, because they rebuild on cores that would otherwise stay idle: with three workers, MICRORAMA serves $1.98\times$ as many requests on the Intel machine and $1.24\times$ as many on AMD. Running the merges in short pieces on the request thread instead, as a level does when it shares a thread with other levels (Section 3.1), lowers throughput by about 16%, because a merge cut into pieces loses the cache locality of a merge run whole. [Tianyao: The loss from running merges in pieces is attributed to locality without counter evidence; the extra carry probes also contribute. Explain why the gain from background workers is smaller on AMD.]

Pre-routing. Pre-routing, which prepares each merge before its carry arrives (Section 3.1), raises throughput where rebuilding limits it. On the AMD machine, with only writes, which keep the workers rebuilding as fast as they can, it raises throughput by up to $1.31\times$ with $B = 4$ and $1.07\times$ with $B = 8$. It also raises the throughput of lookups by 10 to 20% when the request thread runs every merge itself, because merges are published sooner and lookups probe fewer tables. With background workers and only lookups, the workers are mostly idle, and pre-routing makes no difference. [Tianyao: Pre-routing was measured on AMD only, with one pair of runs per configuration; the $B = 8$ gains are near the run-to-run noise.]

Bottleneck without batching. With one request at a time, lookups limit MICRORAMA, and rebuilding does not. At 2^{27} records, the rebuild work of one request takes about as long as its lookup on the Intel machine, but it is spread over the 14 cores that the request thread leaves free, so it keeps only about one of them busy. A lookup, in contrast, runs on a single thread and waits for its probes of the largest tables (Section 5.2). [Tianyao: Measure how much of a lookup these probes take.]

Bottleneck with batching. Batching overlaps the probes of many requests, so it raises the lookup throughput and puts more pressure on rebuilding. The rebuild work per request stays the same, but requests arrive several times faster, and with eight threads serving the batches, only seven cores remain for rebuilding. Whether rebuilding keeps up depends on the machine.

Differences between the machines. The two machines differ in how fast they access memory and how fast they rebuild, and each difference favors one of them. SEV-SNP slows memory more than TDX does: it raises memory latency from 128 to about 205 ns and lowers MICRORAMA's throughput at 2^{27} records by 39% with one request at a time, whereas TDX costs at most 18%. One request at a time, which waits for memory, therefore runs faster on the Intel machine, although the AMD machine is faster without the TEEs.

Batches overlap these memory accesses, so SEV-SNP costs them only 11%, and the cost of rebuilding decides instead. Without VP2INTERSECT and with slower byte permutations, the Intel machine spends $2.6\times$ as long rebuilding per request. Its batches therefore keep about four of the seven rebuild cores busy, and rebuilding limits them: $B = 4$, which rebuilds less per request but probes more tables, is faster than $B = 8$ there, and slower on AMD. As a result, the AMD machine serves batches $1.6\times$ faster. [Tianyao: Report the workers' backpressure waits in the batch rows to show the rebuild limit directly, and measure the SEV-SNP overhead of the tree construction. The Intel virtual machine sees only about 10 to 30 MiB of its host's 504 MiB last-level cache; revisit the Methodology.]

TABLE 3: Tables probed per lookup, memory, and write capacity by branching factor B, with 2^{27} records. The predictions of Table 1 are in parentheses, with memory in 16-byte slots. Averages are measured on the AMD machine, and the other columns on the Intel machine. Write capacity uses one worker per level.

		Tables per lookup			Write
B	Levels	Average	Worst	Memory (GiB)	capacity
2	18	21.1 (40.5)	40 (54)	18.2 (18.0)	—
4	9	16.1 (24.8)	30 (27)	11.8 (11.7)	3.33 M/s
8	7	14.0 (15.9)	20 (18)	9.7 (9.6)	1.72 M/s

Comparison with the idealized model. The idealized model of Table 1 predicts the storage of MICRORAMA to within 2%, but it overestimates the tables per lookup and underestimates the rebuild speed that MICRORAMA needs (Table 3). A lookup probes fewer tables than predicted because MICRORAMA's smallest hash table already holds about 2^9 records, so MICRORAMA has fewer levels than the model, which counts levels down to a single record. The rebuild speed departs more. Section 3.1 predicts that each level needs to rebuild only 3% faster than its average rate, but on the Intel machine, no request waits for a merge only while the workers can absorb writes about $3\times$ as fast as they arrive. The model divides rebuild bandwidth freely among levels, whereas in MICRORAMA three workers serve seven levels, so a level waits whenever its worker is busy with another level's merge. [Tianyao: Confirm the level-one size and the explanation of the worst case (the measured count includes residual tiers, which the model omits), and verify the rebuild-speed explanation with one worker per level.]

[Tianyao: Measure B=2's write capacity, B=4's on AMD, and throughput for each B with the same total cores.]

Choice of branching factor. These costs favor $B = 8$. Compared with $B = 2$, it cuts the tables a lookup probes by a third and the memory by nearly half. It needs more rebuild work, but even so, the workers absorb writes faster than one request thread issues requests on both machines.

6. Conclusion

MICRORAMA uses vectorized bulk rebuilding to change the tradeoff between rebuild work and online probes. Because rebuilds become cheap, a larger branching factor can trade extra rebuild work for fewer tables on the lookup path. Background workers run these rebuilds while requests continue, and the larger branching factor helps here as well: it keeps the levels below the top small, so holding old and

new tables at the same time costs less storage than in prior deamortization schemes. For large entries, a complementary tree construction uses our OMAP as its position map and keeps application payloads out of the hierarchical rebuilds. On a 16-core AMD SEV-SNP server, MICRORAMA serves a 1.5 GiB map of small entries with an average response time of 1.3 μs and no rebuild pauses, achieves 9.7 to 14.1$\times$ the throughput of H2O2RAM where both can be measured, and for 1 KiB entries achieves 12.8$\times$ the throughput of SONIC with an oblivious position map. [Tianyao: Add the memory comparison with H2O2RAM at matched allocations.]

References

[1] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *IEEE S&P*, 2015, pp. 640–656.

[2] J. Demme, R. Martin, A. Waksman, and S. Sethumadhavan, "Side-channel vulnerability factor: A metric for measuring information leakage," in *ISCA*, 2012, pp. 106–117.

[3] L. Domnitser, N. Abu-Ghazaleh, and D. Ponomarev, "A predictive model for cache-based side channels in multicore and multithreaded microprocessors," in *MMM-ACNS*, 2010, pp. 70–85.

[4] O. Goldreich, "Towards a theory of software protection and simulation by oblivious RAMs," in *STOC*, 1987, pp. 182–194.

[5] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious RAMs," *J. ACM*, vol. 43, no. 3, pp. 431–473, 1996.

[6] W.-K. Lin, E. Shi, and T. Xie, "Can we overcome the $n \log n$ barrier for oblivious sorting?" in *SODA*, 2019, pp. 2419–2438.

[7] P. Mishra, R. Poddar, J. Chen, A. Chiesa, and R. A. Popa, "Oblix: An efficient oblivious search index," in *IEEE S&P*, 2018, pp. 279–296.

[8] A. Tinoco, S. Gao, and E. Shi, "EnigMap: External-memory oblivious map for secure enclaves," in *USENIX Security*, 2023, pp. 4033–4050.

[9] R. Ostrovsky, "Efficient computation on oblivious RAMs," in *STOC*, 1990, pp. 514–523.

[10] L. Zheng, Z. Zhang, W. Dong, Y. Zhang, Y. Wu, and C. Wang, "H2O2RAM: A high-performance hierarchical doubly oblivious RAM," in *USENIX Security*, 2025, pp. 8501–8520.

[11] R. Ostrovsky and V. Shoup, "Private information storage (extended abstract)," in *STOC*, 1997, pp. 294–303.

[12] M. T. Goodrich, M. Mitzenmacher, O. Ohrimenko, and R. Tamassia, "Oblivious RAM simulation with efficient worst-case access overhead," in *CCSW*, 2011, pp. 95–100.

[13] P. Williams, R. Sion, and A. Tomescu, "PrivateFS: A parallel oblivious file system," in *ACM CCS*, 2012, pp. 977–988.

[14] E. Stefanov, M. van Dijk, E. Shi, T.-H. H. Chan, C. W. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: An extremely simple oblivious RAM protocol," *J. ACM*, vol. 65, no. 4, pp. 18:1–18:26, 2018.

[15] X. S. Wang, T.-H. H. Chan, and E. Shi, "Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound," in *ACM CCS*, 2015, pp. 850–861.

[16] L. Ren, C. W. Fletcher, A. Kwon, E. Stefanov, E. Shi, M. van Dijk, and S. Devadas, "Constants count: Practical improvements to oblivious RAM," in *USENIX Security*, 2015, pp. 415–430.

[17] K. E. Batcher, "Sorting networks and their applications," in *AFIPS SJCC*, 1968, pp. 307–314.

[18] G. Asharov, T.-H. H. Chan, K. Nayak, R. Pass, L. Ren, and E. Shi, "Bucket oblivious sort: An extremely simple oblivious sort," in *SOSA*, 2020, pp. 8–14.

[19] V. Ramachandran and E. Shi, "Data oblivious algorithms for multicores," in *SPAA*, 2021, pp. 373–384.

[20] N. Ngai, I. Demertzis, J. Ghareh Chamani, and D. Papadopoulos, "Distributed & scalable oblivious sorting and shuffling," in *IEEE S&P*, 2024, pp. 4277–4295.

[21] A. Park, W.-K. Lin, and E. Shi, "NanoGRAM: Garbled RAM with $\widetilde{O}(\log N)$ overhead," in *EUROCRYPT*, 2023, pp. 456–486.

[22] G. Asharov, E. Eiluz, I. Komargodski, and W.-K. Lin, "MegaBlocks: Breaking the logarithmic I/O-overhead barrier for oblivious RAM," in *ACM CCS*, 2025, pp. 4692–4706.

[23] E. Kushilevitz, S. Lu, and R. Ostrovsky, "On the (in)security of hash-based oblivious RAM and a new balancing scheme," in *SODA*, 2012, pp. 143–156.

[24] B. Pinkas and T. Reinman, "Oblivious RAM revisited," in *CRYPTO*, 2010, pp. 502–519.

[25] L. Zheng, "H2O2RAM implementation," 2026, commit 906eb97. [Online]. Available: https://github.com/55199789/H2O2RAM

[26] S. Patel, G. Persiano, M. Raykova, and K. Yeo, "PanORAMa: Oblivious RAM with logarithmic overhead," in *FOCS*, 2018, pp. 871–882.

[27] G. Asharov, I. Komargodski, W.-K. Lin, K. Nayak, E. Peserico, and E. Shi, "OptORAMa: Optimal oblivious RAM," *J. ACM*, vol. 70, no. 1, pp. 4:1–4:70, 2023.

[28] K. G. Larsen and J. B. Nielsen, "Yes, there is an oblivious RAM lower bound!" in *CRYPTO*, 2018, pp. 523–542.

[29] I. Komargodski and W.-K. Lin, "A logarithmic lower bound for oblivious RAM (for all parameters)," in *CRYPTO*, 2021, pp. 579–609.

[30] G. Asharov, I. Komargodski, and Y. Michelson, "FutORAMa: A concretely efficient hierarchical oblivious RAM," in *ACM CCS*, 2023, pp. 3313–3327.

[31] J. Ghareh Chamani, I. Demertzis, D. Papadopoulos, C. Papamanthou, and R. Jalili, "GraphOS: Towards oblivious graph processing," *Proc. VLDB Endow.*, vol. 16, no. 13, pp. 4324–4338, 2023.

[32] N. Talur and I. Demertzis, "SONIC: Concurrent oblivious RAM & data structures for low-latency and high-throughput," in *USENIX Security*, 2026, pp. 4561–4580.

[33] T. Gu, Y. Wang, A. Tinoco, B. Chen, K. Yi, and E. Shi, "Flexway O-Sort: Enclave-friendly and optimal oblivious sorting," in *USENIX Security*, 2025, pp. 7563–7582.

[34] D. Towner, *Intel Advanced Vector Extensions 512 (Intel AVX-512): Permuting Data Within and Between AVX Registers*, Intel Corporation, Oct. 2022, Technology Guide 742407-001US.

[35] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, "Verifying constant-time implementations," in *USENIX Security*, 2016, pp. 53–70.

[36] Intel Corporation, "Data operand independent timing instructions," Feb. 23, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/resources/data-operand-independent-timing-instructions.html

[37] ——, "Data operand independent timing instruction set architecture (ISA) guidance," Sep. 10, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/data-operand-independent-timing-isa-guidance.html

[38] A. Abel and J. Reineke, "uops.info: Latency, throughput, and port usage information for instructions on recent x86 microarchitectures," accessed Sep. 26, 2026. [Online]. Available: https://uops.info/

[39] M. T. Goodrich and M. Mitzenmacher, "Privacy-preserving access of outsourced data via oblivious RAM simulation," in *ICALP*, 2011, pp. 576–587.

[40] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, "Cache-oblivious algorithms," in *FOCS*, 1999, pp. 285–297.

[41] E. Dauterman, V. Fang, I. Demertzis, N. Crooks, and R. A. Popa, "Snoopy: Surpassing the scalability bottleneck of oblivious storage," in *SOSP*, 2021, pp. 655–671.

[42] G. Asharov, I. Komargodski, W.-K. Lin, and E. Shi, "Oblivious RAM with worst-case logarithmic overhead," *J. Cryptol.*, vol. 36, no. 2, p. 7, 2023.

[43] E. Shi, T.-H. H. Chan, E. Stefanov, and M. Li, "Oblivious RAM with $O((\log N)^3)$ worst-case cost," in *ASIACRYPT*, 2011, pp. 197–214.

[44] C. Gentry, K. A. Goldman, S. Halevi, C. Jutla, M. Raykova, and D. Wichs, "Optimizing ORAM and using it efficiently for secure computation," in *PETS*, 2013, pp. 1–18.

[45] "Signal private contact discovery implementation," https://github.com/signalapp/ContactDiscoveryService-Icelake.

[46] Ethereum Foundation, "Merkle Patricia trie," https://ethereum.org/developers/docs/data-structures-and-encoding/patricia-merkle-trie/.

[47] Ethereum Portal Network, "Execution state network specification," https://github.com/ethereum/portal-network-specs/blob/master/legacy/state/state-network.md.

[48] A. Adiletta and B. Sunar, "Spill the beans: Exploiting CPU cache side-channels to leak tokens from large language models," arXiv:2505.00817, 2025. [Online]. Available: https://arxiv.org/abs/2505.00817

[49] S. Sasy, A. Johnson, and I. Goldberg, "Fast fully oblivious compaction and shuffling," in *ACM CCS*, 2022, pp. 2565–2579.

[50] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *CADE*, 2021, pp. 625–635.

Appendix A. Residual Handling Details

One stash per stage. Our concentration bound uses independent or negatively correlated overflows, but the loads of consecutive stages are positively correlated. Every record in a bucket after a stage was in one of the two buckets that the stage merged, so a crowded part of the network overflows at several consecutive stages. An overflowing bucket also passes a full 64 records to the next stage, which makes overflow there more likely. We therefore give each stage its own stash and combine the stages with a union bound, which needs no assumption about how they depend on each other. Within a stage, overflow still concentrates. Because capping only removes records, the overflow of a stage is at most the total excess over 64 of the same stage's loads in an *uncapped* network, which routes the same records by the same tags and never removes one. When the routed records have distinct keys, these loads are negatively correlated multinomial counts with mean at most 32, so a Chernoff bound controls their total excess. In our default configuration, each stage's stash holds one or two 64-slot buckets. Additional stash-tail plots appear in the online full version.

One bucket per range. Because the two tables meet only in the sweep, one 64-slot bucket per range suffices, which keeps the sweep cheap. Every record that overflows during routing belongs to the older table, so the number of residuals in a range depends only on that table's keys and tags. A record that overflows at an early stage has used only the low bits of its tag, so the higher bits that select its range are still uniformly random. In our default configuration, we bound the probability that some range overflows within 2^{32} accesses by about $2^{-70.6}$ for the merges below the top.

Partitioning residuals by range. To collect residuals by range, we partition them recursively by destination bits. Each split uses the order-preserving oblivious compaction network of Sasy et al. [49], executed in the byte-sliced layout. The partitions form a binary tree whose leaves are the range buckets. Each node's public capacity is the smaller of the total input capacity and the sum of its leaves' capacities. Because a node contains only records destined for those leaves, it fits whenever they all do. Bounding the number of residuals per range therefore suffices for every intermediate partition as well.

The depth-first traversal reuses temporary buffers and can pause between splits in their fixed public order (Section 3.1). Buffer optimizations appear in the online full version.

The sweep. Each carry bucket already holds exactly the records for the output buckets with the same low tag bits, and the remaining bits select each record's destination among those buckets. The carry's stored tags lack these remaining bits, so the sweep recomputes them in registers by hashing each key. It therefore reads the carry in place instead of retagging a full temporary copy. To insert records from the carry's tier, the sweep scans it directly after the range bucket: in our default configuration, the tier holds at most two buckets.

Size of the residual tier. Recovering overflowed records makes the residual tier as small as any placement of the table's keys into 64-slot buckets can make it. A residual remains after the sweep only if its destination bucket is full. That bucket stays full, because the sweep removes a main-table copy only when it has a newer copy of the key to insert, and it fills every slot that it frees. When the sweep ends, a bucket to which the new table's hash assigns X distinct keys therefore holds $\min(X, 64)$ of them, and the tier holds the excess $\max(X - 64, 0)$ of every bucket. Any placement of the same keys into 64-slot buckets under the same hash, including one computed by sorting, leaves at least this many records outside the main table. Even so, a full top of 2^{22} buckets needs more than 64 tier slots with probability above 2^{-62}, so the top's tier needs two buckets.

Tombstones at the top. A merge below the top keeps every tombstone, because the tombstone may still hide an older copy of its key in a larger level, or protect a hash under which a table has already received a real probe for the key (Section 3.4). The top consumes carries in order, and each of its merges replaces the old top, which retires the older hashes that a tombstone in the carry protects. The top can therefore discard tombstones, but only after they have taken effect in its merge: an erasure of x must still remove the old top's copy of x, from the main table and from the residuals, before the merge drops the tombstone.

Some tombstones can be dropped as soon as the top reads the carry. A tombstone is *unshadowed* when the lookup that appended it missed in every table of its probe plan, including the online stash and the residual tiers, so that no older copy of its key exists. Finding an existing tombstone does not establish this. The top omits unshadowed tombstones before they occupy output slots, so requests for absent keys do not consume the nominal capacity $N = 32T$, which counts live keys. The carry itself stays intact for earlier readers (Section 3.4).

The sweep drops the remaining tombstones before it tests a bucket's capacity. Otherwise a record could overflow into the final residual tier although a tombstone that is dropped later frees a slot in its bucket, and the tier would exceed the excess $\max(X - 64, 0)$ above, with X counting live keys. For each destination bucket, the sweep therefore first drops the older residuals that the carry replaces or erases, removes the old top's copies of the keys in the carry's tier, and drops every tombstone. Only then does it merge the carry into the bucket, so every record that it places is the newest copy of a live key.

Failure probability. The checker of Section 2.4 bounds each capacity under ideal hash functions, and each bound holds for any request sequence fixed in advance. It adds these bounds by a union bound over the first capacity test that fails, so it requires no independence between them.

Appendix B. Deamortization Analysis

This appendix derives the idealized costs that Table 1 reports for MICRORAMA and for prior deamortization schemes, and gives the configuration details of Section 3.

Idealized probe model. The rebuild speeds and average table counts of MICRORAMA in Table 1 come from an idealized model of one non-top level. The model isolates the effect of the branching factor and of carries that remain separately queryable until they are absorbed. One incoming-carry interval is one time unit. The first carry is adopted at zero cost. Merging a carry into a partial of k carry-sized units costs $k+1$ abstract work units. The model assumes that preparation dominates this cost: the k units that route the partial execute before the carry arrives, and only one unit, the carry's share, must follow it. In MICRORAMA, the work after arrival is residual partitioning and the sweep, a single pass over the output table, whereas preparation passes the partial through every stage of the network. [Tianyao: Report the measured share of each merge's time that follows the carry's arrival, per level and carry ordinal, against the model's $1/(k+1)$.] Rebuild bandwidth is perfectly divisible, and one later carry may wait while a merge finishes. A second waiting carry stalls the model. This abstraction omits the construction of bottom carries from the online stash, top compaction, residual work, reader retirement, and competition among actual worker phases.

Closed-form rebuild speed. We measure speed per non-top level in the binary baseline's work units. For power-of-two $B \geq 2$, let

$$r_B = \min\left\{B, \left\lceil \frac{\sqrt{16B+17}-1}{2} \right\rceil \right\}.$$

The exact minimum speed is

$$s_B^* = \max\left\{ \frac{B+1}{2} - \frac{1}{B}, B + \frac{5}{2} - \frac{r_B}{2} - \frac{2(B+1)}{r_B} \right\}. \quad (1)$$

The first term is the cycle's average work. For the second, define a carry's *publication delay* as the time from its arrival to the publication that removes it as a separate table from the lookup path. Every merge at rebuild speed s leaves a publication delay of at least $1/s$. The last m merges then require $s \geq [1+m(2B-m+1)/2]/(m+2)$ to finish before a second waiting carry arrives. Maximizing over $0 \leq m \leq B-2$ gives $m = r_B - 2$. The resulting bound is attainable under the idealized schedule. [Tianyao: Check that the streamable model's minimum speeds are attainable under the implementation's coupled region lifetimes; the table reports the model, not measured throughput.]

Closed-form average tables per lookup. Set $s = s_B^*$, $h = \lfloor s \rfloor$, and $t = B - h$. The steady-state average number of queryable tables per non-top level is

$$\bar{q}_B^* = \begin{cases} 3/2, & B = 2, \\ 11/6, & B = 4, \\ 1 + \dfrac{1}{B} + \dfrac{1}{s} + \dfrac{t(t+1)(3h+t+2-3s)}{6Bs}, & B \geq 8. \end{cases} \quad (2)$$

To obtain this average, let d_k be the publication delay of carry k in a cycle, and count one queryable table for one carry interval as a table-time unit. The current table contributes $B-1+d_{B-1}-d_0$ such units, and the unincorporated carries contribute $\sum_{k=0}^{B-1} d_k$. For $B \geq 8$, the periodic schedule has $d_0 = 1$, $d_1 = 2/s$, $d_k = 1/s$ for $2 \leq k < h$, and

$$d_{h+j-1} = \frac{1 + j(2h+j+1)/2}{s} - j, \qquad 1 \leq j \leq t.$$

The last delay is two; summing the table lifetimes and dividing by B gives Eq. (2). For $B = 2, 4$, the delay vectors are $(0, 1)$ and $(5, 4, 7, 14)/9$. The total average is $\bar{q}_B^* \log_B N$, the average entry of Table 1. At $B = 8$, $s_8^* = 9/2$ and $\bar{q}_8^* = 127/72$ exactly. Thus the total is $(127/216)\log N \simeq 0.59 \log N$, or $127/108 \simeq 1.18$ times the amortized average of $\frac{1}{2}\log N$.

For the worst case, count tables by the level whose regions store them. Each level holds at most two published tables, the current partial and the preceding outgoing table, as Figure 5 shows for $B = 4$. The rebuild destination is not yet queryable. Incoming carries are counted at the preceding level, so each published table is counted once. When rebuilding meets its deadlines, the worst-case total is therefore $2 \log_B N$. Dividing by the amortized binary worst case of $\log N$ queryable tables gives $(2/\log B)\times$ overhead, or about $0.67\times$ for $B = 8$.

The tradeoff at higher rebuild speeds and its infinite-speed limit appear in the online full version.

Prior-work accounting. The table counts exclude cache and stash scans and count a cuckoo table or Bloom-filter table as one table, regardless of its internal probe cost. For the two-slot schedule of Ostrovsky and Shoup [11, Section 5], let α be the fraction of rebuild work available before the second carry. Retaining sources until publication requires $s \geq \max\{1.5, 6(1-\alpha)\}$ under the regular schedule. A speed of $1.5\times$ would require crediting at least 75% early work. Their construction chooses the hash function and starts rehashing in advance, but gives no early-work fraction or pre-routing procedure that leaves only our final carry-absorption pass. We use $\alpha = 0$ as a conservative comparison convention, charging the full rebuild after the second carry. This gives $s = 6$, a queryable lifetime of $4/3$ carry intervals, and $5/6$ queryable tables per level, or $5/3 \simeq 1.67$ times the amortized average. The peak must account for phase offsets between levels. With incoming interval $\tau_\ell = 2^\ell$, the first carry arrives at $a_\ell = 4(\tau_\ell - 1)/3$; the two carries remain queryable on $[a_\ell, a_\ell + 4\tau_\ell/3)$ and $[a_\ell + \tau_\ell, a_\ell + 4\tau_\ell/3)$, repeated every $2\tau_\ell$. Summing these intervals gives a peak of $L+1$ over L regular levels, not $2L$, because the two-table phases cannot all coincide. Thus the same $s = 6$ schedule gives $1.67\times$ average and $1\times$ leading worst-case overhead in tables per lookup. [Tianyao: Cite the report or GO96-analysis/check_prior_work.py for the phase proof and event-simulation check, or state the proof here.]

Goodrich et al.'s prescribed $2\times$ schedule alternates between one and two published tables per regular height, giving an average of 1.5.

For PD-ORAM, fix publication at the end of each m-request period, as in its construction. An odd period reshuffles one size-m generation, and an even period merges two into a size-$2m$ output. If that rebuild costs $C(2m)$, its deadline requires rate $C(2m)/m$, at least twice the matched amortized binary-generation rate $C(2m)/(2m)$. We report this deadline lower bound, not PD-ORAM's upper bound on extra work from postponing deletions. Input-size inflation can add further work. The fixed publication periods give 1.5 queryable tables

per height on average and a simultaneous peak of two, hence table-count ratios 3 and 2.

Prior-work storage accounting. Let H be the physical size of the largest table in each scheme, $S_0 \simeq 2H$ the table storage of its amortized version, and M the record slots that it reserves. Table 1 places every hash table at load factor one half, where $H \simeq 2N$ and $S_0 \simeq 4N$ record slots. For Ostrovsky and Shoup [11, Section 5], fix two table banks per level, $\alpha = 0$, $s = 6$, and Goldreich and Ostrovsky's rehash procedure [5, Section 5.5]. For a level whose table occupies W slots, the procedure first copies its inputs, of physical capacities $W/2$ and W, into a work array. It later pads the D input records to $2D$ real and dummy records and adds W empty slots, so the array needs $W+2D$ slots. At load factor one half, $D = W/2$, so the array holds $2W$ slots. Sorting compacts records within this array but does not remove its allocation. The finished table occupies its first W slots, which can trivially be the under-construction bank itself, so only the remaining W slots are additional workspace. Two banks and this workspace therefore reserve $3W$ per level, giving $M \geq 3S_0 \simeq 12N$ before metadata. The workspace is used only by the rebuild and never holds a published table, so the two-slot lifetime constraint used for $s = 6$ is unchanged. We retain a fixed workspace at each level and assume no cross-level scratch pooling or alternative compact-array rebuild.

Goodrich et al.'s layout [12, Section 3] reserves $H + 2\sum_{j\geq 1} H/2^j \simeq 3H$ for published tables and $\sum_{j\geq 0} H/2^j \simeq 2H$ for rebuild destinations. Thus the arrays require $5H/S_0 \simeq 2.5\times$. For cuckoo expansion $c = 2(1+\epsilon)$ physical slots per logical entry, where Goodrich et al. require only a small positive constant ϵ, the history list adds H/c record slots, raising the ratio to at least $2.5 + 1/(2c)$. The expansion does not change the prescribed publication schedule or the tables per lookup. In absolute terms, $H \simeq cN$, so the arrays and history list hold $(5c+1)N$ record slots, more than $11N$ for every $c > 2$. Table 1 reports the limit $11N$, or $2.75\times$ the $4N$ baseline, as the load approaches one half.

In PD-ORAM's full-period schedule [13, Section 5.3], an even generation at height i retains two size-W_i source tables while constructing a size-$2W_i$ replacement. These even periods can end simultaneously at all regular heights. Immediately before publication, the source and replacement reservations total $4\sum_{i<L} W_i$, and the old largest table of size H is still queryable. The largest replacement is already included in the sum and is not counted again. Since $\sum_{i<L} W_i \simeq H$, this gives $M \geq 5H \simeq 2.5S_0 \simeq 10N$. The hash expansion and Bloom-filter parameters are fixed across generations and in the denominator. Delete logs and filter-construction scratch are additional. [Tianyao: Cite the report that verifies the simultaneous phase from the generation recurrence.]

Storage in physical slots. For uniform branching factor B, let T be the top's bucket count and M_{b} the total number of main-table buckets. Summing the levels' regions and dropping lower-order terms gives

$$\frac{M_{\mathrm{b}}}{2T} \approx 1 + \frac{3B-1}{2B(B-1)}.$$

At $B = 8$, the ratio is $1.21\times$. Each main bucket contributes 64 physical slots, so when logical capacity matches nominal capacity, $N = 32T$, the main-slot count is $64M_{\mathrm{b}} = 2(M_{\mathrm{b}}/T)N$. This conversion includes bucket slack but still excludes residual tiers, metadata, and scratch. Capacity tuning for other input sizes appears in the online full version.

Computing a shared hash. Each routing hash applies AES under the secret key to the key together with a public pair, the level and carry ordinal of the last carry that uses the hash. To compute the pair, start from level ℓ and carry ordinal v, and follow the forwarding step whenever ℓ is non-top and v is a multiple of its branching factor B_ℓ: replace (ℓ, v) with $(\ell+1, v/B_\ell)$ and repeat. The terminal pair names the hash. For uniform $B = 4$ and at least three levels, $(1, 16)$ forwards to $(2, 4)$ and then to $(3, 1)$. All three publications therefore use the pair $(3, 1)$, which each producer can compute before its consumer runs. The same rule applies to mixed factors because each step uses that level's public B_ℓ.

Partial tables of non-power-of-two size. A partial table with jU_ℓ buckets need not be rounded up to the next power of two. For power-of-two U_ℓ, factor $j = r2^e$ with odd r. Routing uses r blocks of 2^eU_ℓ buckets, followed by a terminal r-way split when $r > 1$. The extra work of this split lets the table keep the exact partial sizes used in Figure 5. For example, a seven-unit partial occupies seven units instead of eight, saving one eighth of the padded main storage.

Appendix C. Machine-checked Guarantees

The correctness and security of MICRORAMA rest on three claims. No stash, range bucket, or tier may overflow within 2^{32} accesses except with probability below 2^{-64} (Section 2.4), no key may receive two real probes under one routing hash, and every lookup must return the newest value of its key (Section 3.4). Our checker bounds each capacity on its own and assumes that the merges handle their records as described. The other two claims have so far rested on arguments in prose, even though one hash serves a sequence of tables, a batch can repeat a key, and the top discards tombstones. We therefore wrote a model of MICRORAMA in the Lean proof assistant [50], whose kernel checks every step of a proof, and proved all three claims for the model.

The model. The model is a reference implementation of MICRORAMA written as Lean functions. It builds the carries from the online stash, runs the merges below the top with their stage stashes, range buckets, and sweeps, prepares and merges the top, and initializes the map. Every merge checks each capacity and reports the first one that is exceeded. The model runs the merges one after another, as a single thread would, and uses the parameters of our default configuration with 2^{27} records and of its variant with branching factor eight at every level. Its hash functions are ideal: each routing hash assigns every key an independent, uniformly random 64-bit word, and each dummy probe reads a fresh random word. The request sequence is an input fixed before the words are drawn, so the client is non-adaptive, as in our checker.

No capacity overflows. For every request sequence fixed in advance, the model exceeds some capacity within the first 2^{32} accesses with probability at most 2^{-69}, and its first overflow falls in any later window of 2^{32} accesses with probability at most 2^{-64}. Over K windows, the model therefore overflows with probability at most $2^{-69} + (K-1)2^{-64}$. The proof follows our checker: it bounds the capacities of each merge

under ideal hashing and adds the bounds by a union bound over the first test that fails (Appendix A). Its numerical budgets are rounded conservatively, so the bound is slightly looser than the checker's $2^{-69.7}$. Beyond the arithmetic, the proof establishes facts that the checker takes for granted, for example that the set of keys each merge receives is determined by the request sequence alone, whatever the hash words.

The access pattern hides the requests. The second theorem formalizes the security argument of Section 3.4 for the hash tables. The adversary sees the bucket that each request, or each request of a batch, touches in every hashed table of its probe plan, together with the sequence of probe plans, which records every publication. The online stash and the residual tiers add nothing to this view, because every access scans them in full. For every request sequence, and every schedule of probe plans that orders its tables from newest to oldest as Section 3.4 requires, the view is within $2^{-69} + (K-1)2^{-64}$ of what a simulator produces from the schedule, the batch boundaries, and the type of each request alone. Without an overflow, the two distributions are equal. Two request sequences that agree on this public information thus produce views within twice this distance.

The proof establishes that no key receives two real probes under one hash for every request sequence, including repeated keys within a batch, carries that keep their hash across publications, and tombstones at the top. Every real probe therefore reads its own uniformly random word, and every dummy probe reads a fresh one. The view is then a fixed function of distinct entries of a uniform tape, which gives it exactly the simulator's distribution, and the capacity theorem bounds the probability that MICRORAMA departs from this view.

Lookups return the newest value. For every request sequence and every choice of the hash words, each lookup in the model returns the value of the newest record that the initialization or an earlier request wrote for its key, or reports the key absent if this record is a tombstone or does not exist. In a batch, the earlier requests include those before the lookup in the same batch. The proof formalizes the freshness order of Section 3.4, including the tombstones that the top discards. It also shows that these lookups touch exactly the buckets of the view above, so the second theorem covers the accesses that return these values.

Matching the model to the engine. A theorem about the model says nothing about MICRORAMA unless the engine computes what the model computes. For the bottom level's carry construction, we prove in Lean that a transcription of the native algorithm, over 64-bit words and bit masks, produces exactly the model's carry. For everything else, a replay harness runs the engine, records every table it publishes and every bucket each access touches, and compares them with the model evaluated in Lean on the same requests. On 64 runs covering both of our configurations, branching factors 2, 4, and 8, initialization, batches, two rebuild schedules, and forced overflows at every kind of merge, 7,053 published tables match slot by slot, and each forced overflow fails in the model at the same capacity. Across these runs, the touched buckets and the real or dummy choice match the model at every one of more than 5.3 million table probes, both when merges run in pieces between accesses and when every merge runs whole before the next access. This evidence covers the runs we executed, not every request sequence.

Assumptions. Besides the non-adaptive client, the theorems replace AES with ideal hash functions, and we do not formalize the distance between the two. They cover keys below 2^{27}, the addresses of an ORAM with 2^{27} records. The view contains only the buckets of the hash tables, so the rebuilds' own memory accesses, which depend only on public sizes (Section 2.2), and all timing lie outside it. The schedule of probe plans and the record that each request writes are inputs to the model. That the engine derives the schedule from public counters is checked by the replay, not proved. The replay also confirms the premise about unshadowed tombstones in Appendix A. [Tianyao: Decide whether the threat model of Section 3.4 should state these assumptions instead, with a reference here.]

Checking the proofs. AI agents wrote the proofs against a separately frozen model and theorem statements. A script checks that each theorem has its frozen statement and uses only Lean's standard three axioms, and the Lean kernel rechecks every proof. The development workflow appears in the online full version.